

COMPUTER GRAFICA

Appunti di [Malchiodi Riccardo](#)

IMPORTANTE: questi appunti sono stati fatti con l'aiuto di vari LLM. Questo corso è meglio seguirlo in presenza e prendere appunti a lezione, purtroppo non riuscendo a seguire tutte le lezioni ho dovuto arrangiarmi. Questo documento può risultare utile come ripasso veloce e qualche spiegazione dei primi laboratori.

Event Driven Programming

Evento

Un evento è un segnale inviato dall'utente al programma. Può essere asincrono (può arrivare in qualunque momento). Può essere caratterizzato da quale dispositivo proviene o da parametri (quale tasto per esempio).

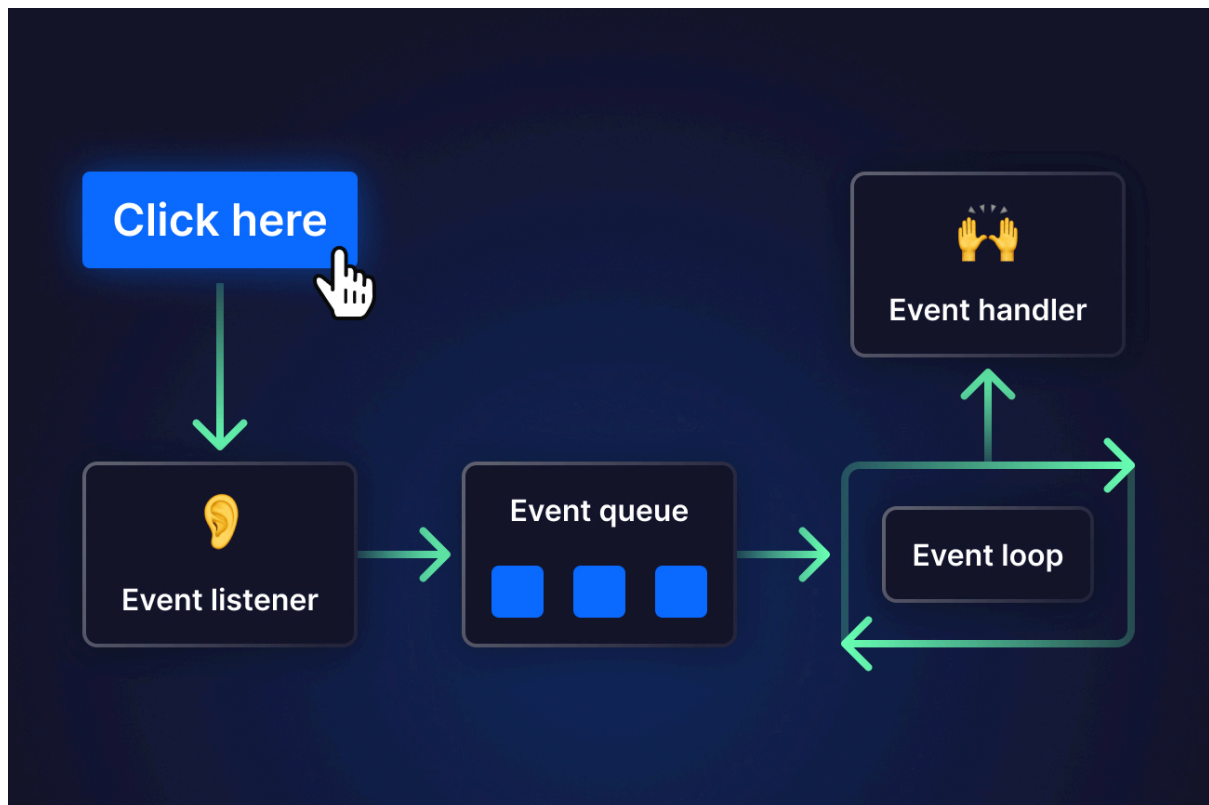
Event Listener e coda

L'event listener fa parte del sistema operativo, è un processo che resta in ascolto in attesa di eventi. Appena arriva un evento lo aggiunge in coda.

La coda degli eventi mantiene e consuma gli eventi in ordine di arrivo (fifo). Fa da "ponte" tra il sistema operativo e il programma, ogni programma ha la sua coda.

Event Loop e Event Handler

Entrambi fanno parte del programma, **l'event loop** è un processo/ciclo che controlla continuamente lo stato della coda. Se trova un evento lo estrae e lo passa all'event handler. **L'event handler** reagisce a un evento svolgendo le operazioni correlate al verificarsi dell'evento stesso. Le operazioni dipendono da: stato del programma, quale evento si è verificato e i parametri che caratterizzano l'evento.



Window Management System

Fa parte del sistema operativo, implementa l'ambiente Desktop. Si frappona fra le applicazioni (dotate di interfaccia utente) e i dispositivi di I/O per gestire l'interazione:

- **Input:** eventi dell'utente verso l'applicazione
- **Output:** feedback dall'applicazione verso l'utente (testo e grafica)

Gestisce tutte le finestre di tutte le applicazioni (che ne hanno).

Risorse e Richieste

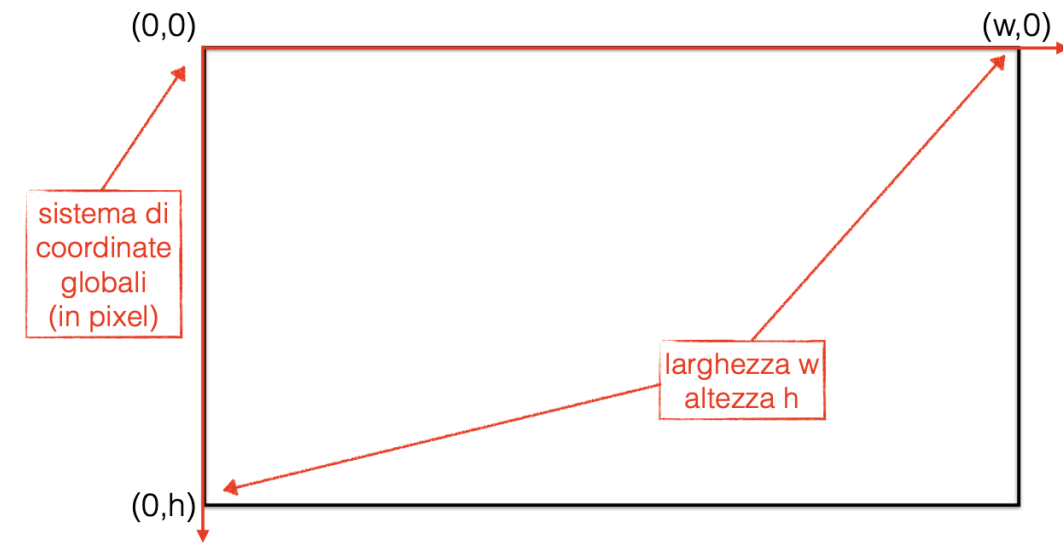
Un'applicazione interattiva ha bisogno di risorse per poter:

- produrre output
- ricevere input (eventi)

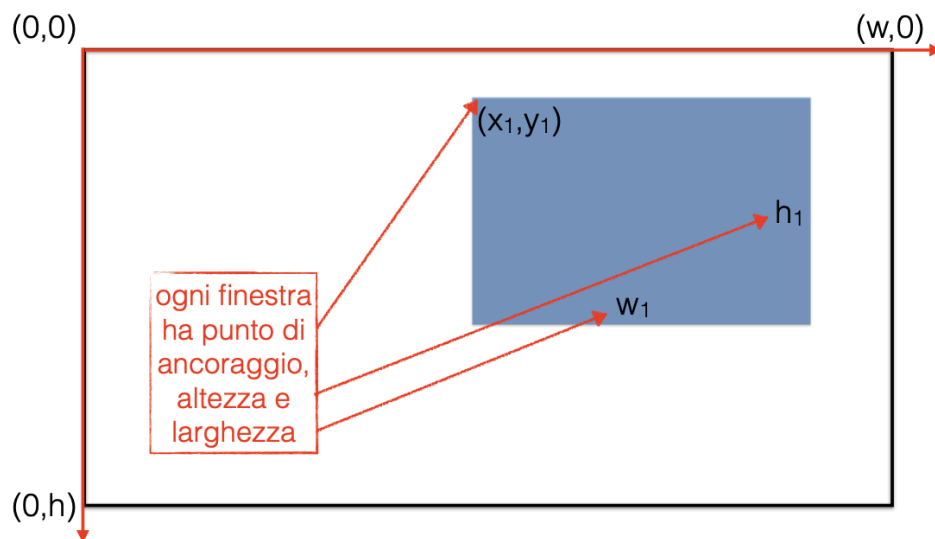
Una risorsa è per esempio una bitmap, una finestra, font, ecc ecc.

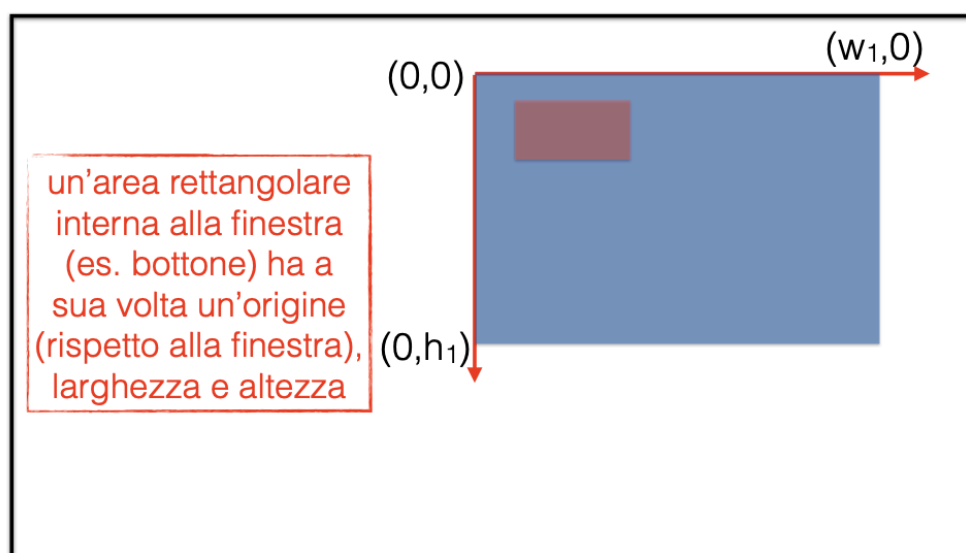
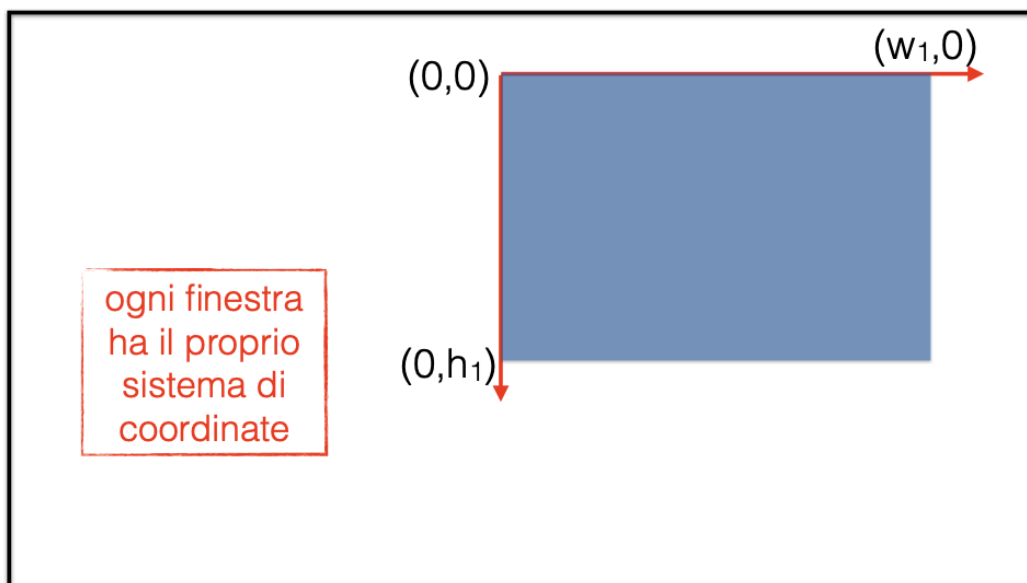
Una richiesta è un atto con cui l'applicazione chiede una risorsa al WMS. Una volta ottenuta, una risorsa è accessibile tramite un identificatore.

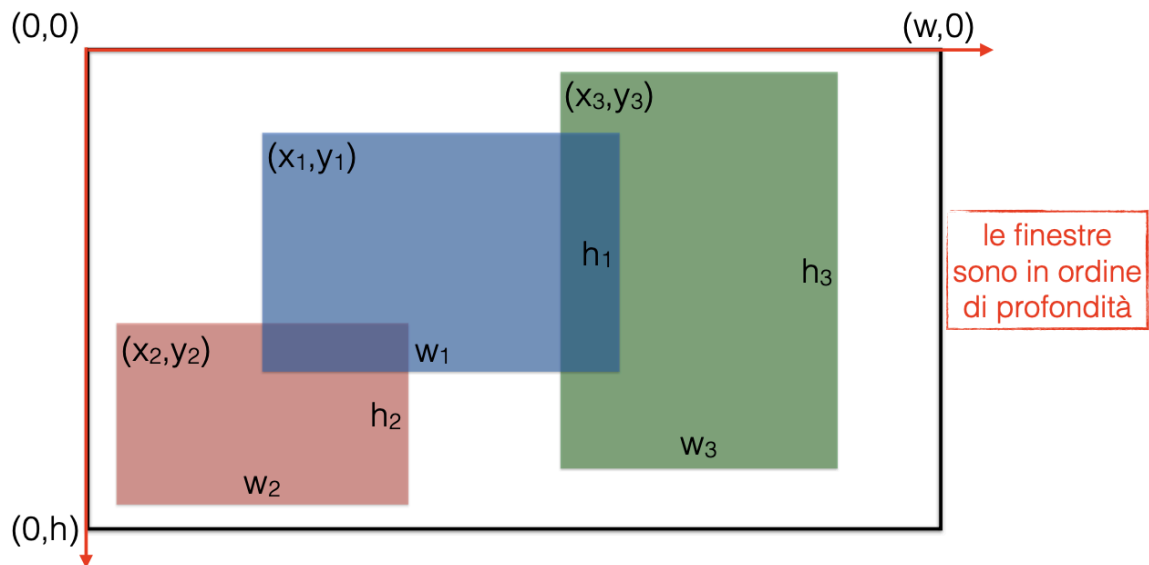
Coordinate schermo



Finestre







Finestre

Ogni finestra è un'area autonoma di I/O come uno schermo nello schermo. Una finestra fa riferimento a un'applicazione. Un'applicazione può usare una o più finestre. Una finestra è caratterizzata da una serie di informazioni come:

- ancoraggio e dimensioni
- visibile/invisibile
- caratteristiche grafiche
- contenuto grafico

Eventi e finestre

Ogni evento è generato in una finestra. Una sola finestra ha il focus per ogni determinato evento. Il focus può essere fissato dalla posizione del cursore del mouse o dal click. Ogni evento generato dall'utente si riferisce alla finestra che ha il focus per quel tipo di evento.

Funzioni callback

Sono funzioni che sono registrate rispetto al verificarsi di eventi. Quando si verifica un evento di un certo tipo, il programma manda in esecuzione la callback registrata per quel tipo di evento. Il modo di implementare le callback dipende dal sistema specifico che si utilizza per mettere in comunicazione l'applicazione col Window Management System.

Toolkit

Libreria ad alto livello per la gestione di finestre ed eventi. Permette di costruire una Graphical User Interface(GUI) utilizzando costrutti “preconfezionati”:

- bottoni
- cursori
- menu
- campi di test
- maschere
- decori

Finestre in SFML

`sf::RenderWindow` è una classe che fornisce window sulle quali si può disegnare in SFML. Una window è caratterizzata da:

- Risorse video: dimensioni in pixel w e h, profondità in bit-per-pixel
- Titolo(stringa), Stile, Stato
- Eventuali informazioni di contesto

Le operazioni principali sono:

- creazione: l'applicazione chiede al WMS di fornirgli una window
- close: chiusura definitiva della window e delle sue risorse
- display: mostrare a schermo il contenuto grafico della window
- clear: cancellazione del contenuto grafico
- gestione del focus
- pollEvent, waitEvent: gestione degli eventi

Esempio di finestra in SFML

```
#include <SFML/Graphics/RenderWindow.hpp>

int main()
{
    auto window = sf::RenderWindow(sf::VideoMode({800u, 600u}), "CMake SFML Project")
    while (window.isOpen())
    {
        while (const std::optional event = window.pollEvent())
        {
            if (event->is<sf::Event::Closed>())
            {
                window.close();
            }
        }
        window.clear();
        window.display();
    }
}
```

`sf::Event` è il tipo generico per qualunque evento. Ci sono poi sotto-tipi per eventi specifici:

- **Closed** evento di chiusura finestra
- **KeyPressed/KeyReleased** è stato premuto/rilasciato un tasto
- **MouseButtonPressed/MouseButtonReleased**
- **MouseMoved**

Abbiamo due modi per implementare event loop e event handler:

1. Modo classico

- L'event loop è un ciclo while che fa polling sulla coda degli eventi
- A ogni ciclo, estrae un evento dalla coda, ne riconosce il sotto-tipo ed esegue le operazioni previste per quel tipo di evento
- L'event handler è implementato direttamente dentro il loop

Schema del modo classico:

- ➔ si estrae un evento generico `event` tramite `pollEvent`
- ➔ in una cascata di if, ci si chiede di che tipo di evento si tratta:
 - ◆ `event->is<T>()` è una funzione Booleana che dice se l'evento è di tipo T. Si usa se non serve altro che il tipo di evento per trattarlo
 - ◆ `event->getIf<T>()` è una funzione che restituisce i dati dell'evento se questo è di tipo T. Si usa se i dati dell'evento sono rilevanti(ad esempio tasto premuto o

movimento del mouse)

2. Tramite callback

- si definisce una funzione callback per ogni tipo di evento
- event loop e event handler sono implementati da un'unica chiamata alla funzione **handleEvents**
 - riceve come parametri la lista delle callback
 - estende in successioni eventi della cosa, finché ce ne sono
 - per ogni tipo di evento, applica la callback opportuna

schema del modo a callback:

- si scrivono diverse funzioni void **handle** ciascuna con un solo parametro del tipo di evento da trattare
- si passa il controllo degli eventi alla funzione membro **handleEvents** che riceve come parametro un'espressione lambda con template automatico sul tipo di evento:

```
window.handleEvents( [] (const auto &event)
                    { handle(event); });
```

Di solito le funzioni handle devono modificare lo stato del programma, quindi devono poter accedere alle **variabili di stato**. La soluzione facile (e sporca) è lo stato è una variabile globale. Una soluzione più pulita è che lo stato è una variabile **gs** del main e viene passata come ulteriore parametro alle funzioni handle:

```
gs.window.handleEvents( [&] (const auto &event)
                        { handle(event, gs); });
```

Computer Grafica 2D

Grafica in SFML

La **sf::RenderWindow** è la destinazione dell'output grafico. Abbiamo 3 comandi principali:

1. **window.clear()** cancellazione della finestra, ovvero colorata con un colore uniforme
2. **window.draw()** disegna qualcosa ma non si vede ancora
3. **window.display()** visualizza nella window tutto ciò che è stato disegnato

Frame buffer

è un'area di memoria dedicata a mantenere le informazioni sui pixel che formano l'immagine. è una sorta di **copia virtuale della finestra**. SFML usa il **double buffering**. Nella window è visualizzato il contenuto di un buffer. Il comando **draw** disegna in un altro buffer nascosto. Il comando **display** scambia di ruolo i due buffer, in modo da visualizzare il contenuto di quanto appena disegnato. Il ciclo di disegno è delimitato da **clear** e **display**. Nel mezzo ci possono essere molti comandi **draw** che insieme contribuiscono a generare l'immagine della scena. L'immagine viene costruita virtualmente nel buffer nascosto e visualizzata di colpo quando si chiama **display** in modo da evitare effetti “sfarfallio”.

Sistemi di coordinate

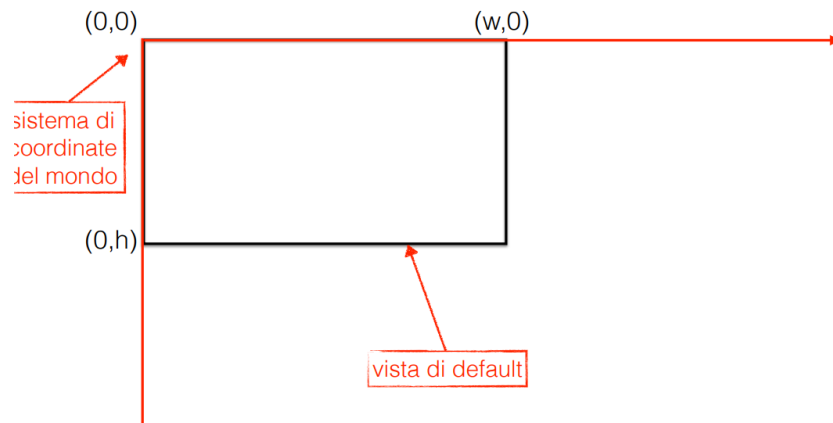
Gli oggetti grafici vengono disegnati posizionandoli nel piano in un sistema di coordinate detto **sistema del mondo**. Il sistema del mondo è virtualmente infinito. La window rappresenta una vera e propria “finestra sul mondo” che ci mostra una porzione limitata del piano. Possiamo disegnare cose ovunque nel sistema del mondo, ma solo quelle comprese nella regione inquadrata dalla finestra saranno visibili. Di default, il sistema di coordinate del mondo e quello della finestra **coincidono**: origine in alto a sinistra; asse x verso destra; asse y verso il basso; unità di misura in **pixel**.

Per il momento ci limitiamo al default dove:

- l'origine del mondo è nell'angolo in alto a sinistra della finestra
- la porzione di mondo visibile nella finestra va da (0,0) a (w,h) dove w e h sono rispettivamente la larghezza e l'altezza della finestra in pixel

NOTA: se la finestra viene ridimensionata, la porzione di mondo visibile non viene automaticamente aggiornata alle nuove dimensioni, ma resta la stessa.

Sistemi di coordinate



Primitive Grafiche

Cosa possiamo disegnare:

- **Testo:** con font, stile, dimensioni
- **Forme semplici:** rettangoli, cerchi, poligoni convessi / stellati
- **Sprite:** rettangoli + texture
- **Vertex array:** forme più complesse ottenute come collezioni di triangoli o segmenti.

Per ogni primitiva c'è una classe, che contiene tutti i metodi ammessi per quel tipo di primitiva. **Prima** si stabiliscono tutte le caratteristiche della primitiva da disegnare: posizione, grandezza, colore, stile ecc.

Poi si disegna la primitiva nella finestra tramite il metodo draw della classe **RenderWindow**

Text

Classe sf::Text

Il suo costruttore è:

Text(const Font &font, String string="", unsigned int characterSize=30)

Esistono metodi **set** (per impostare) e **get** (per leggere) relativi a diverse proprietà fondamentali, tra cui:

- il font utilizzato
- la stringa (contenuto testuale)
- il colore e la dimensione del carattere
- la spaziatura e la dimensione dell'area occupata dal testo

Oltre agli attributi estetici esistono anche metodi specifici per definire come il testo si colloca nel mondo grafico:

- `setOrigin(Vector2f origin)`: serve a stabilire il punto di ancoraggio del testo; indica quale punto della scritta deve essere considerato l'origine per le trasformazioni
- `move(Vector2f offset)`: sposta il punto di ancoraggio di un certo valore rispetto alla sua posizione attuale

Font

Classe `sf::Text`

Va caricato in genere da disco. Ad esempio possiamo fare:

```
sf::Font font("arial.ttf")
```

Si specifica alla creazione del testo o successivamente tramite il metodo apposito.

- `sf::Text text(font);`
- `text.setFont(font);`

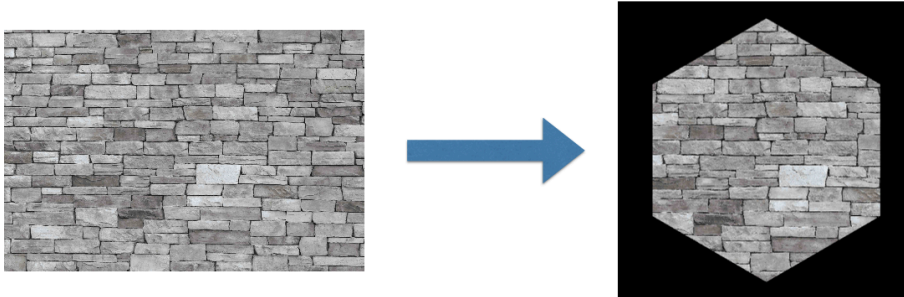
Shape

Vediamo alcune forme semplici:

- `sf::RectangleShape rectangle({120.f, 50.f});`
- `sf::CircleShape circle(200.f);`
- `sf::CircleShape circle(80.f, 3);` triangolo
- `sf::CircleShape circle(80.f, 4);` quadrato
- `sf::CircleShape circle(80.f, 4);` ottagono

Texture

Una texture è un'immagine digitale che viene utilizzata per "colorare" l'interno di primitive geometriche.



La texture deve essere caricata (di solito da disco):

```
sf::Texture texture("image.png")
```

Se ne può caricare anche solo una parte:

```
sf::Texture texture("image.png", false , sf::IntRect({10,10},{32,32}));
```

Le texture vengono caricate in **memoria video**. Questo significa che sono veloci da disegnare ma lente da consultare/aggiornare. Spostare informazione tra memoria primaria/secondaria e memoria video è un'operazione lenta o addirittura lentissima. Conviene caricare le texture necessarie in memoria video all'inizio. Le texture occupano spazio, se ne servono tante potrebbero eccedere la capacità della memoria video: in questo caso conviene caricarle a batch in momenti in cui il programma ha il tempo di farlo (es: passaggio tra livelli in un videogame). Una texture può contenere diversi "pezzi" usati per colorare entità.

Codice di esempio

```

int main()
{
    sf::ContextSettings settings;
    settings.antiAliasingLevel = 8;
    auto window = sf::RenderWindow(sf::VideoMode({800u, 600u}), "CMake SFML Project", sf::Style::Default, sf::State::Windowed, settings);
    window.setFramerateLimit(30);

    sf::Texture texture("/Users/iraxty/Desktop/Computer-Grafica/resources/1856-51-stone-texture-wall-charcoal-color.jpg");

    while (window.isOpen()) // main loop
    {
        while (const std::optional event = window.pollEvent())
        { // event loop + event handler
            if (event->is<sf::Event::Closed>())
            {
                window.close();
                break;
            }
        }

        // Do graphics
        window.clear();

        float sqrt32 = sqrt(3.f) / 2.f;
        sf::ConvexShape s(6);
        s.setPoint(0, {sqrt32, -0.5f});
        s.setPoint(1, {0.f, -1.f});
        s.setPoint(2, {-sqrt32, -0.5f});
        s.setPoint(3, {-sqrt32, 0.5f});
        s.setPoint(4, {0.f, 1.f});
        s.setPoint(5, {sqrt32, 0.5f});
        // s.setFillColor(sf::Color::Green);
        s.setScale(200.f, 200.f);
        s.setPosition({400.f, 300.f});
        s.setTexture(&texture);
        s.setTextureRect(sf::IntRect({0, 0}, {(int)(sqrt32 * 2000), 2000}));
        // std::cout << "BBBox: (" << s.getGlobalBounds().position.x << ", "
        // << s.getGlobalBounds().position.y << ", "
        // << s.getGlobalBounds().size.x << ", "
        // << s.getGlobalBounds().size.y << ")\n";
        window.draw(s);
        window.display();
    }
}

```

Output:



Spiegazione codice

1. **Setup:** ci creiamo una finestra grafica di **800 x 600** pixel. Impostiamo l'antialiasing a **livello 8**, questo lo facciamo per smussare le scalettature e fissare il limite di aggiornamento a **30FPS**.
2. **Event loop:** Gli eventi sono l'intercettazione delle azioni dell'utente, in questo caso solo la richiesta di chiusura della finestra tramite `sf::Event::Closed`
3. **Geometria e rendering:** Il `window.clear()` pulisce lo schermo dal frame precedente ad ogni ciclo. Il `sf::ConvexShape` definisce un esagono regolare creando un poligono convesso a 6 lati. Abbiamo una serie di `setPoint`.
 - `s.setPoint(0, {sqrt32, -0.5f})`: definisce il vertice 0, si trova in alto a destra
 - `s.setPoint(1, {0.f, -1.f})`: definisce il vertice 1, e la punta superiore dell'esagono

- **s.setPoint(2, {-sqrt32, -0.5f}):** Definisce il vertice 2, si trova in alto a sinistra
- **s.setPoint(3, {-sqrt32, 0.5f}):** Definisce il vertice 3 si trova in basso a sinistra
- **s.setPoint(4, {0.f, 1.f}):** Definisce il vertice 4, è il punto più in basso
- **s.setPoint(5, {sqrt32, 0.5f}):** Definisce l'ultimo vertice, completa la chiusura del poligono tornando verso la parte destra

Il comando **s.setScale({200.f, 200.f});** applica un fattore di scala di 200 sia in larghezza che in altezza. Siccome i punti erano molto piccoli (-1, 1) questo comando ingrandisce la forma rendendola visibile.

s.setPosition({400.f, 300.f}) posiziona il centro della forma alle coordinate (400, 300) della finestra, ovvero al centro di una finestra **800 x 600**.

s.setTexture(&texture) assegna alla forma la texture caricata precedentemente.

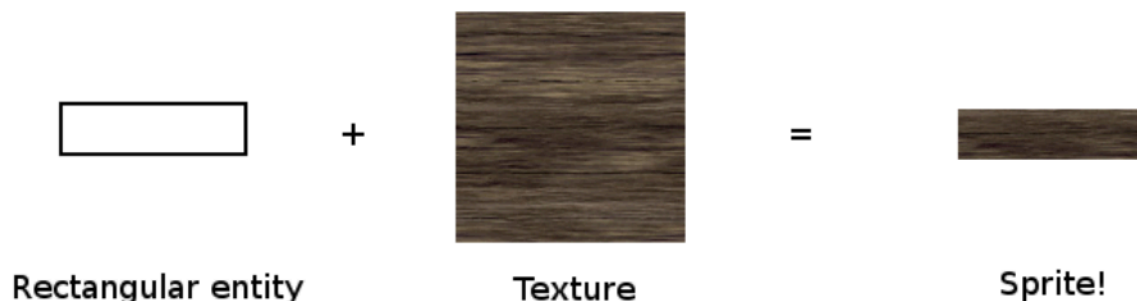
L'immagine verrà proiettata sulla superficie dell'esagono.

s.setTextureRect(sf::IntRect({0, 0}, {(int)(sqrt32 * 2000), 2000})); Definisce quale porzione della texture visualizzare. Prende un rettangolo che parte da (0,0) con una larghezza calcolata e un'altezza di 2000 pixel. Serve a mappare correttamente i pixel dell'immagine sui vertici.

Sprite

Uno sprite è costituito da un rettangolo e una texture che lo riempie.

sf::Sprite sprite(texture)



Di default, l'estensione dello sprite è quella in pixel della texture associata.

Metodi principali:

- **posizionamento:** `setPosition( {x, y} );`
- **colore:** `setColor( c )`
- **punto di aggancio/origine in coordinate locali:** `setOrigin( {x0, y0} );`
- **ridimensionamento:** `setTextureRect(sf::IntRect({x,y},{w,h}));`
- **spazio occupato nel mondo:** `getGlobalBounds();`
- **trasformazioni geometriche:** `move, rotate, scale` ecc

MainSprite.cpp

```
#include <SFML/Graphics.hpp>

#include <vector>
#include <string>
#include <math.h>
#include <iostream>

int main()
{
    sf::ContextSettings settings;
    settings.antiAliasingLevel = 8;
    auto window = sf::RenderWindow(sf::VideoMode(800u, 600u), "CMake SFML Project", sf::Style::Default, sf::State::Windowed, settings);
    // window.setFramerateLimit(30);
    sf::Texture texture("./resources/1856-51_stone-texture-wall-charcoal-color.jpg");

    while (window.isOpen()) // main loop
    {
        while (const std::optional event = window.pollEvent())
        { // event loop + event handler
            if (event->is<sf::Event::Closed>())
            {
                window.close();
                break;
            }
        }

        // Do graphics
        window.clear();

        sf::Sprite sprite(texture);
        sprite.setTextureRect(sf::IntRect(300, 200, 500, 300));
        //sprite.setPosition(200.f, 100.f);
        //sprite.scale(0.5f, 0.5f);
        //sprite.setColor(sf::Color::Red);
        //sprite.rotate(sf::degrees(60));
        window.draw(sprite);

        window.display();
    }
}
```

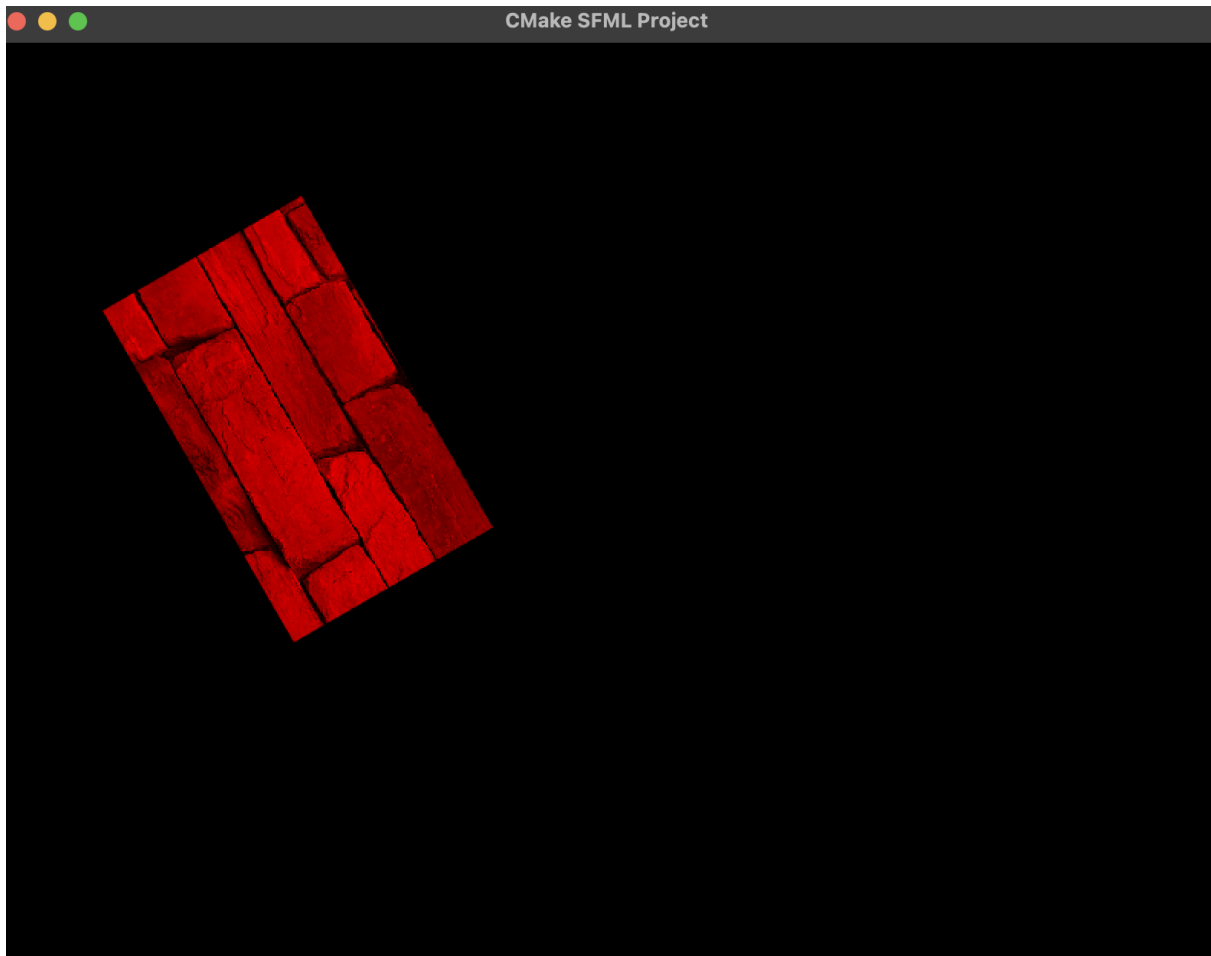
In questo caso l'unico evento disponibile nell'event loop è la chiusura della finestra. La creazione dello sprite si ha con: `sf::Sprite sprite(texture)`. Andiamo a selezionare una parte della texture tramite `sprite.setTextureRect(sf::IntRect({300, 200}, {500, 300}));`

dove:

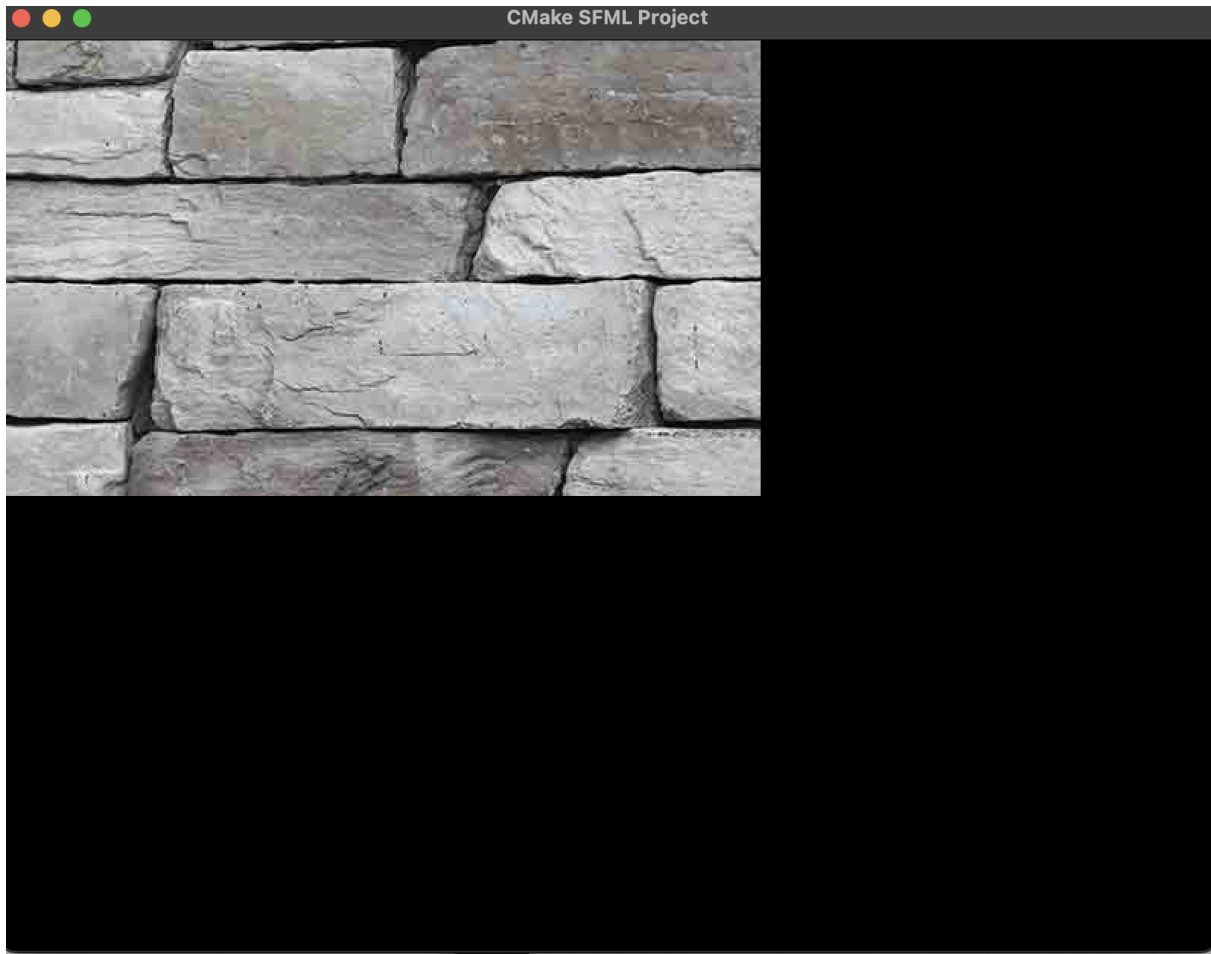
- `({300,200})` = punto iniziale nella texture
- `({500,300})` = larghezza e altezza

Quindi prende un rettangolo con larghezza = 500, altezza = 300 e con l'origine (300, 200)

Il codice commentato avrebbe prodotto come output:



mentre il codice spiegato sopra:



`main_animated_sprite.cpp`

```

#include <SFML/Graphics.hpp>

#include <vector>
#include <string>
#include <math.h>
#include <iostream>

int main()
{
    sf::ContextSettings settings;
    settings.antiAliasingLevel = 8;
    auto window = sf::RenderWindow(sf::VideoMode(800u, 600u), "CMake SFML Project", sf::Style::Default, sf::State::Windowed, settings);
    window.setFramerateLimit(10);
    sf::Texture texture("../resources/SpaceInvaders.png");
    bool swap = false;
    sf::Vector2f pos(<0.f, 0.f>), step(<5.f, 0.f>);
    int sign = 1;

    while (window.isOpen()) // main loop
    {
        while (const std::optional event = window.pollEvent())
        { // event loop + event handler
            if (event->is<sf::Event::Closed>())
            {
                window.close();
                break;
            }
        }

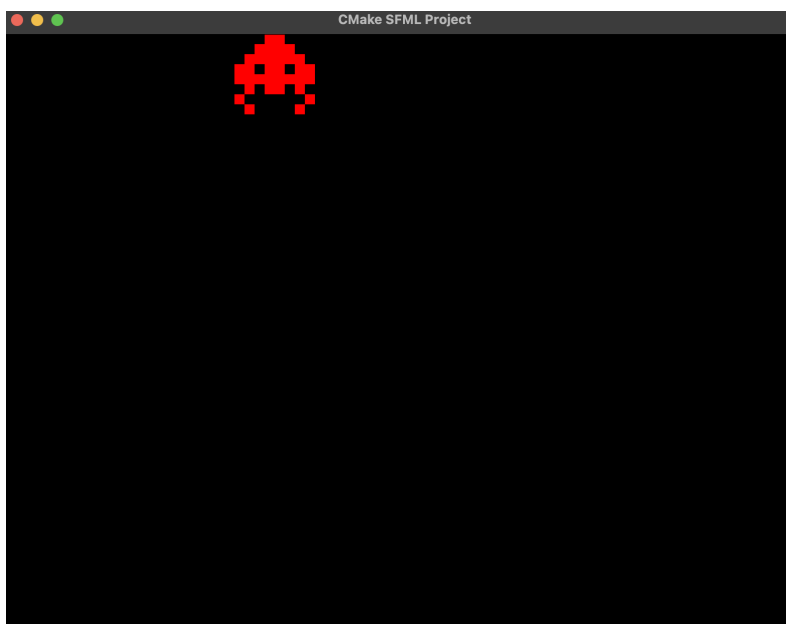
        // Do graphics
        window.clear();

        sf::Sprite sprite(texture);
        sprite.scale(<10.f, 10.f>);
        sprite.setColor(sf::Color::Red);
        sprite.setPosition(pos);
        unsigned sw = sprite.getLocalBounds().size.x;
        if (swap)
            sprite.setTextureRect(sf::IntRect(<5, 1>, <8, 8>));
        else
            sprite.setTextureRect(sf::IntRect(<5, 11>, <8, 8>));
        swap = !swap;
        if (<(pos.x >= window.getSize().x - 80 && sign == 1) || (pos.x <= 0 && sign == -1)>)
        {
            pos += <0.f, 30.f>;
            sign = -sign;
        }
        else
            pos += <step.x * sign, step.y>;
        window.draw(sprite);

        window.display();
    }
}

```

output (animato):



Questa volta mettiamo come limite 10FPS, questo lo facciamo per avere un'animazione fluida (per questo tipo di sprite) e di evitare la CPU di lavorare troppo (inutilmente nel nostro esercizio).

Variabili per animazione e movimento:

```
bool swapFrame = false;
sf::Vector2f pos({0.f, 0.f}), step({5.f, 0.f});
int sign = 1;
```

SwapFrame alterna i frame dello sprite per simulare il movimento del personaggio. **pos** è la posizione dello sprite di partenza, **step** è l'incremento del movimento in pixel per frame. **Sign** è la direzione del movimento, con 1 stiamo dicendo di andare a destra. Se avessimo messo -1 sarebbe stato verso sinistra.

Lo sprite viene creato con:

```
sf::Sprite sprite(texture);
sprite.scale({10.f, 10.f});
sprite.setColor(sf::Color::Red);
sprite.setPosition(pos);
```

scale ingrandisce lo sprite di 10 pixel. **setColor** gli applica il colore e la **setPosition** posiziona lo sprite sulla finestra.

```
if (swap)
    sprite.setTextureRect(sf::IntRect({5, 1},
    {8, 8})))
```

Il **setTextureRect** Seleziona un rettangolo della texture. I valori tra le {} sono i frame del personaggio, questa if ha il compito di alternarli, l'idea sarebbe che il codice alterna continuamente frame1 e frame2. Sarebbe la stessa logica di farlo per esempio in Unity tramite Animator.

la riga **swap = !swap** serve semplicemente per aggiornare il booleano.

Il movimento vero e proprio è dato da:

```
if ((pos.x >= window.getSize().x - 80 && sign == 1) || (pos.x <= 0 && sign == -1))
{
    pos += {0.f, 30.f};
    sign = -sign;
}
```

```
else
    pos += {step.x * sign, step.y};
```

La logica è di controllare se lo sprite ha raggiunto il bordo della finestra. **Se** sì allora scende di 30 pixel ($y += 30$) e cambia la direzione con $sign = -sign$.

Altrimenti si muove di $step.x$ nella direzione corrente ($sign$).

Trasformazioni

Le trasformazioni geometriche permettono di spostare e/o ridimensionare qualsiasi primitiva grafica. Si esprimono coordinate del mondo. Si applicano relativamente al punto di aggancio/origine della primitiva.

Origine della primitiva:

- si trova in un certo punto del mondo, stabilito con `setPosition`.
- si riferisce a un certo punto della primitiva, stabilito con `setOrigin`.

Ad esempio: rettangolo di 100 x 50 pixel ancorato al punto (100,100) del mondo. Il punto di ancoraggio è al centro del rettangolo.

```
sf::RectangleShape r({100.f, 50.f});
r.setPosition({100.f, 100.f});
r.setOrigin({50.f, 25.f});
```

La traslazione sposta la posizione dell'oggetto nel mondo applicando un offset alle due coordinate.

Ad esempio: il centro del rettangolo viene portato nel punto (150, 80) del mondo.

```
sf::RectangleShape r({100.f, 50.f});
r.setPosition({100.f, 100.f});
r.setOrigin({50.f, 25.f});
r.move({50.f, -20.f});
```

Abbiamo l'asse X settato a 100, con `move` ci aggiungiamo 50 (= 150).

Asse Y abbiamo `setPosition` = 100, ma con `move` mettiamo -20 (= 80)

Rotazione: ruota l'oggetto attorno alla sua origine di un determinato angolo.

Ad esempio: ruota di 30 gradi in senso antiorario attorno al centro del rettangolo.

Se non avessimo spostato l'origine al centro, la rotazione avverrebbe attorno all'angolo in alto a sinistra

```
sf::RectangleShape r({100.f, 50.f});  
r.setPosition({100.f, 100.f});  
r.setOrigin({50.f, 25.f});  
r.rotate(sf::degrees(-30.f));
```

Scalatura: scala l'oggetto attorno alla sua origine dei fattori di scala specificati.

Esempio: raddoppio la lunghezza e dimezzo l'altezza.

```
sf::RectangleShape r({100.f, 50.f});  
r.setPosition({100.f, 100.f});  
r.setOrigin({50.f, 25.f});  
r.scale({2.f, 0.5f});
```

main_transform.cpp

```

#include <SFML/Graphics.hpp>

#include <vector>
#include <string>

int main()
{
    sf::ContextSettings settings;
    settings.antiAliasingLevel = 8;
    auto window = sf::RenderWindow(sf::VideoMode({800u, 600u}), "CMake SFML Project", sf::Style::Default, sf::State::Windowed, settings);

    window.setFramerateLimit(100);
    sf::Vector2f offset({0.f, 0.f});
    auto rot = sf::Angle::Zero;
    while (window.isOpen()) // main loop
    {
        while (const std::optional event = window.pollEvent())
        { // event loop + event handler
            if (event->is<sf::Event::Closed>())
            {
                window.close();
                break;
            }
        }

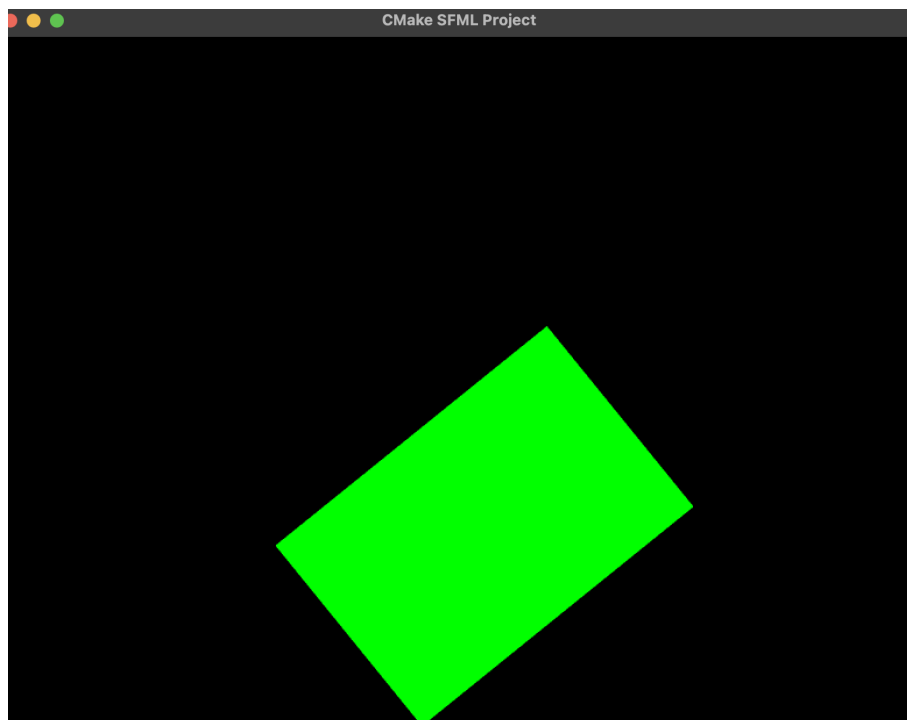
        // Do graphics
        window.clear();

        sf::RectangleShape r({300.f, 200.f});
        r.setOrigin({150.f, 100.f});
        r.setPosition({100.f, 100.f});
        r.move(offset);
        r.rotate(rot);
        r.setFillColor(sf::Color::Green);
        window.draw(r);

        offset += {1.f, 1.f};
        rot += sf::degrees(1.f);
        window.display();
    }
}

```

Output (animato)



Questa volta settiamo un limite di rendering a 100 frame per secondo. Le variabili per movimento e rotazione sono:

```
sf::Vector2f offset({0.f, 0.f});
```

```
auto rot = sf::Angle::Zero;
```

Offset è un vettore 2D che usiamo per spostare la forma.

rot è una variabile di tipo angle inizializzato a zero, servirà a ruotare la forma.

Creazione della forma

```
sf::RectangleShape r({300.f, 200.f});
```

```
r.setOrigin({150.f, 100.f});
```

```
r.setPosition({100.f, 100.f});
```

Il **rectangleShape** crea un rettangolo largo 300px e alto 200px. Il **setOrigin** imposta il punto di origine al centro del rettangolo e il **setPosition** posiziona il rettangolo nella finestra a coordinate (100,100).

Spostamento e rotazione

```
r.move(offset);
```

```
r.rotate(rot);
```

il **move** sposta il rettangolo di offset.x e offset.y dal punto di origine. Il **rotate(rot)** ruota il rettangolo di rot gradi.

Colore e disegno

```
r.setFillColor(sf::Color::Green);
```

```
window.draw(r);
```

setFillColor mi permette di colorare il rettangolo col colore verde, per disegnarlo usiamo la funzione **draw**.

Aggiornamento variabili

```
offset += {1.f, 1.f};
```

```
rot += sf::degrees(1.f);
```

offset cresce di (1,1) quindi il rettangolo si muoverà verso il basso a destra. **rot** aumenta di 1 grado (rotazione continua)

Vertex Array

Modo generale per creare primitive più complesse.

Classe `sf::VertexArray`.

Una primitiva si descrive tramite:

- Un elenco di punti(vertici): classe `sf::Vertex`
- Il tipo di primitiva, che dice come i vertici vengono aggregati per formare entità geometriche

La classe `sf::Array` contiene solo tre attributi pubblici:

- Posizione in coordinate del mondo
- **Colore:** un valore della classe `sf::Color`
- Coordinate di texture

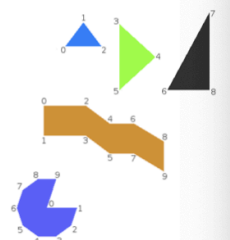
La classe `sf::VertexArray` è un wrapper per:

- Un vector di `sf::Vertex`
- L'indicazione del tipo di primitiva `sf::PrimitiveType`

Fornisce alcuni metodi per accedere e modificare i suoi membri

- Tipi di primitive:

- Punti sparsi (1 pxel): `sf::PrimitiveType::Points`
- Segmenti di linea sparsi (spessore 1 pixel): `sf::PrimitiveType::Lines`
- Linee poigionali (spessore 1 pixel): `sf::PrimitiveType::LineStrip`
- Triangoli sparsi: `sf::PrimitiveType::Triangles`
- Strisce di triangoli: `sf::PrimitiveType::TriangleStrip`
- Ventagli di triangoli: `sf::PrimitiveType::TriangleFan`

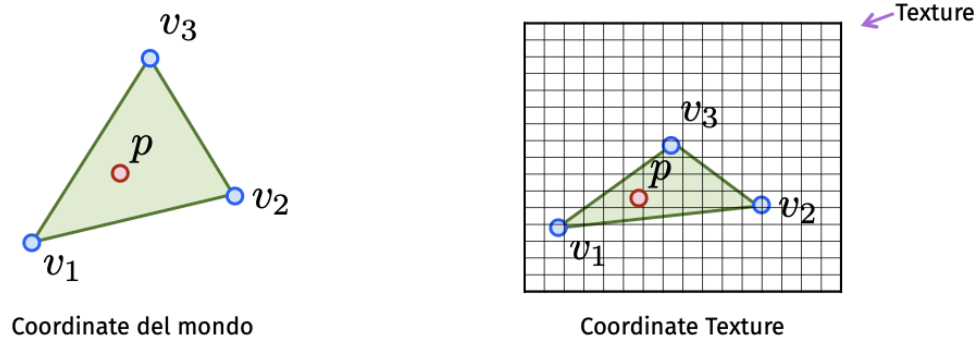


Ogni vertice ha il suo colore. Se i vertici di una primitiva hanno colori diversi, il colore viene interpolato all'interno della primitiva.

Le texture si possono applicare anche ai vertex array. È necessario assegnare a ogni vertice del vertex array una coppia di coordinate texture. Dato in triangolo formato dai vertici v_1 , v_2 , v_3 :

- si considerano le proiezioni di tali vertici sulla texture attraverso le loro coordinate texture
- la porzione di texture compresa nel triangolo formato dalle proiezioni dei tre vertici viene usata per “colorare” il triangolo v_1 , v_2 , v_3

Il colore applicato in un generico punto p del triangolo è quello contenuto nel texel su cui si proietta p



I vertex array funzionano diversamente dalle altre primitive. Le trasformazioni si applicano a tutti i vertici della primitiva e devono essere gestite globalmente. Le trasformazioni si specificano con matrici (classe `sf::Transform`). L'operazione di draw viene eseguita specificando la trasformazione da applicare.

main_vertexArray.cpp

```
////////////////////////////////////
// Headers
////////////////////////////////////
#include <SFML/Graphics.hpp>

#include <vector>
#include <string>
#include <math.h>

int main()
{
    sf::ContextSettings settings;
    settings.antiAliasingLevel = 8;
    auto window = sf::RenderWindow(sf::VideoMode({800u, 600u}), "CMake SFML
Project", sf::Style::Default, sf::State::Windowed, settings);
    window.setFramerateLimit(30);

    sf::Texture
texture("../resources/1856-51_stone-texture-wall-charcoal-color.jpg");
    sf::Transform transform(200.f, 0.f, 300.f,
                           0.f, 200.f, 300.f,
                           0.f, 0.f, 1.f);

    sf::RenderStates states;
```

```

states.transform = transform;
states.texture = &texture;
while (window.isOpen()) // main loop
{
    while (const std::optional event = window.pollEvent())
    { // event loop + event handler
        if (event->is<sf::Event::Closed>())
        {
            window.close();
            break;
        }
    }

    // Do graphics
    window.clear();

    float sqrt32 = sqrt(3.f) / 2.f;
    sf::VertexArray wall(sf::PrimitiveType::TriangleFan, 8);
    wall[0].position = sf::Vector2f(0.f, 0.f);
    wall[1].position = sf::Vector2f(sqrt32, -0.5f);
    wall[2].position = sf::Vector2f(0.f, -1.f);
    wall[3].position = sf::Vector2f(-sqrt32, -0.5f);
    wall[4].position = sf::Vector2f(-sqrt32, 0.5f);
    wall[5].position = sf::Vector2f(0.f, 1.f);
    wall[6].position = sf::Vector2f(sqrt32, 0.5f);
    wall[7].position = sf::Vector2f(sqrt32, -0.5f);
    wall[0].color = wall[1].color = wall[2].color =
        wall[3].color = wall[4].color = wall[5].color =
            wall[6].color = wall[7].color = sf::Color::Green;
    wall[0].texCoords = {1500.f, 1000.f};
    wall[1].texCoords = {sqrt32 * 1000 + 1500, -0.5 * 1000 + 1000};
    wall[2].texCoords = {1500, -1000 + 1000};
    wall[3].texCoords = {-sqrt32 * 1000 + 1500, -0.5 * 1000 + 1000};
    wall[4].texCoords = {-sqrt32 * 1000 + 1500, 0.5 * 1000 + 1000};
    wall[5].texCoords = {1500, 1000 + 1000};
    wall[6].texCoords = {sqrt32 * 1000 + 1500, 0.5 * 1000 + 1000};
    wall[7].texCoords = {sqrt32 * 1000 + 1500, -0.5 * 1000 + 1000};

    window.draw(wall, states);
    window.display();
}
}

```

Come sempre importiamo la nostra Texture. Creiamo un **transform** che sarà la nostra matrice dove verrà disegnata la nostra figura (la nostra griglia sostanzialmente).

Ci dichiariamo `states` passandoli la texture e la griglia.

Il blocco di codice dal window clear in poi si occupa di generare e disegnare un esagono.

Blocco position: viene calcolata la costante $\frac{\sqrt{3}}{2}$ derivata dal seno/coseno di 30° e 60° , necessaria per trovare i vertici di un esagono di raggio 1.

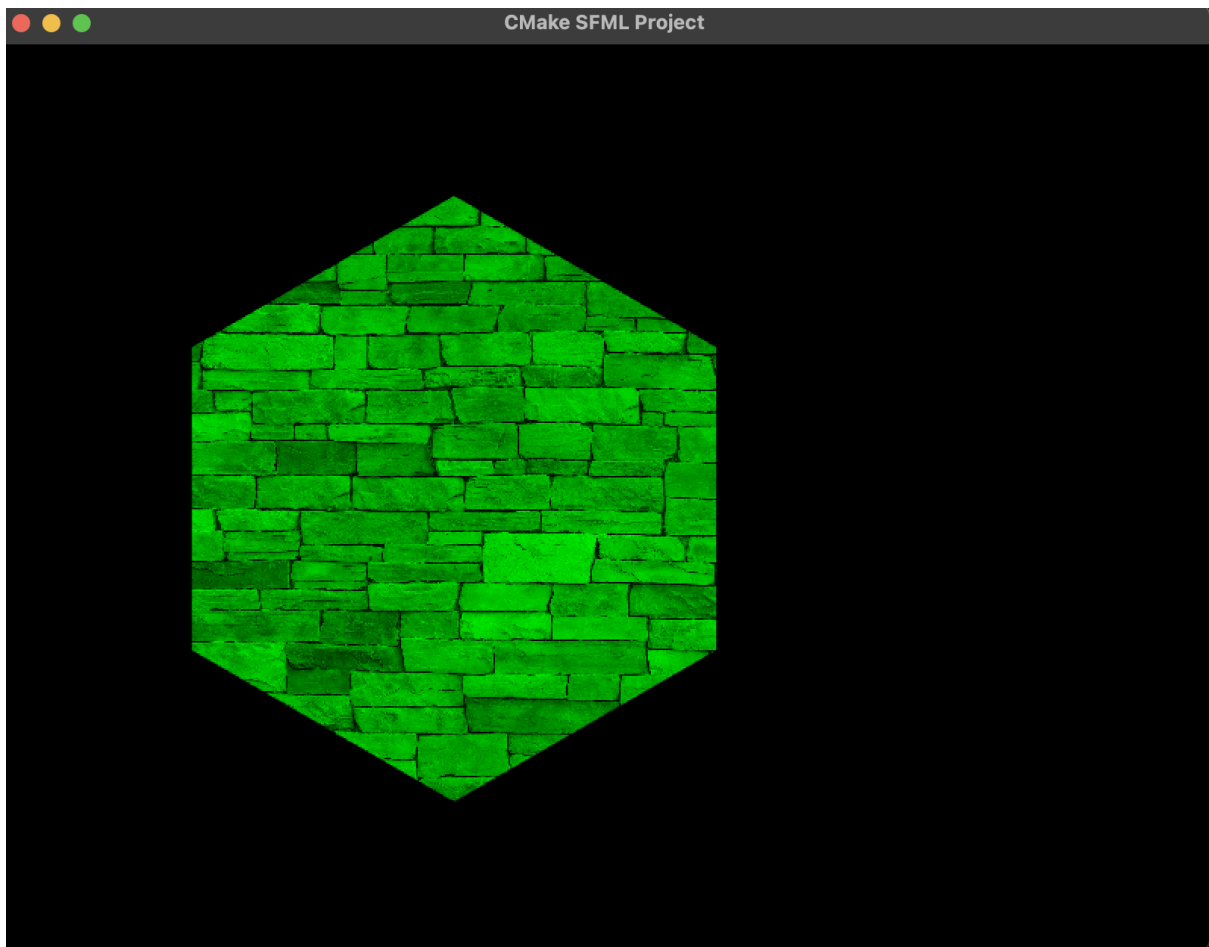
`sf::PrimitiveType::TriangleFan, 8`: Crea una geometria a "ventaglio". Il vertice [0] funge da centro esatto (0.f, 0.f).

I vertici da [1] a [6] compongono i 6 angoli del perimetro. Il vertice [7] replica esattamente le coordinate del vertice [1] per "chiudere" la figura e completare l'ultimo triangolo del ventaglio.

color: tutti i vertici vengono inizializzati al colore verde.

texCoords: adatta la geometria ai pixel reali dell'immagine caricata in memoria. Il centro geometrico viene ancorato al pixel (1500, 1000) dell'immagine. Le posizioni esterne vengono scalate di un moltiplicatore 1000 per ritagliare esattamente un'area esagonale ampia dall'immagine di partenza

Output:



Viste

Una vista è caratterizzata da:

- Un rettangolo in coordinate del mondo, che costituisce la porzione di scena da visualizzare(vista)
- Un rettangolo in coordinate window che specifica in che porzione della window va visualizzata la scena (viewport)

La vista può traslata/ruotata/scalata che corrisponde a trasformare il primo rettangolo in modo che catturi una diversa porzione del mondo, senza cambiare la scena. **La vista è una proprietà della window.**

Di default:

- il rettangolo di vista ha l'angolo in alto a destra nell'origine del mondo e si estende in coordinate mondo delle unità date da larghezza e altezza della window in pixel, quindi copre il rettangolo [0, w] x [0,h].
- la viewport copre l'intera finestra

Si può definire una vista diversa (`sf::View view`) e poi usarla per modificare la vista della window(`window.setView(view)`).

La vista diventa attiva solo dopo averla associata alla window.

- Il rettangolo di vista si può stabilire alla creazione:

```
sf::View view(sf::FloatRect({200.f, 200.f}, {300.f, 200.f}));
```

crea un rettangolo di vista [200,500]x[200,400] in coordinate mondo, specificando l'angolo in alto a sinistra e le dimensioni

- oppure `sf::View view({350.f, 300.f}, {300.f, 200.f});`

crea lo stesso rettangolo specificandone il centro e le dimensioni.

Il rettangolo di vista **deve avere le stesse proporzioni della viewport** su cui viene mappato, altrimenti la scena viene deformata(stretch). Nel caso più comune, la viewport copre l'intera finestra. Quando si ridimensiona la finestra le proporzioni cambiano ed è necessario aggiornare la vista:

$$\text{larghezza-vista} : \text{altezza-vista} = w : h$$

Le dimensioni w e h della window dopo che è stata ridimensionata sono una proprietà dell'evento di resize. Le coordinate window sono indipendenti dalle dimensioni della window in pixel e mappano l'intera window nel quadrato [0,1] x [0,1]. Una viewport si specifica come porzione di tale quadrato. Ad esempio, se vogliamo limitare la viewport alla metà destra della window, dovremo eseguire:

```
view.setViewport(sf::FloatRect({0.5f,0.f},{0.5f,1.f}));
```

Di solito si usa la viewport di default; altre viewport possono essere utili per creare sotto-finestre.

main_view.cpp

lab1 - textEditor

(controllare repository github per il sorgente)

```
////////////////////////////////////
// Headers
////////////////////////////////////
#include <SFML/Graphics.hpp>

#include <vector>
#include <string>

////////////////////////////////////
/// GUI State
const int FONT_SIZE = 20;

struct State
{
    // General resources
    sf::RenderWindow window;
    const sf::Font font{"../resources/DejavuSansMono-5m7L.ttf"};
    // Text buffer
    std::vector<std::string> log;

    State(unsigned w, unsigned h, std::string title)
    {
        window = sf::RenderWindow(sf::VideoMode({w, h}), title);
        // window.setVerticalSyncEnabled(true); // alternative to setFrameRateLimit
        window.setFramerateLimit(60);
        log.resize(1, "");
    }
};

///
```

```

////////////////////////////////////

////////////////////////////////////
// Callback functions
void handle(const sf::Event::Closed &, State &gs)
{
    gs.window.close();
}

void handle(const sf::Event::TextEntered &textEnter, State &gs)
{
    unsigned last = gs.log.size() - 1;
    // if (textEnter.unicode == 10 || textEnter.unicode == 13)
    if (textEnter.unicode == '\n' || textEnter.unicode == '\r') // enter
        gs.log.emplace_back("");
    else if (textEnter.unicode >= ' ' && textEnter.unicode <= '~') // printable char
    {
        gs.log[last] += static_cast<char>(textEnter.unicode);
    }
}

void handle(const sf::Event::Resized &resized, State &gs)
{
    sf::FloatRect visibleArea({0.f, 0.f}, sf::Vector2f(resized.size));
    gs.window.setView(sf::View(visibleArea));
}

template <typename T>
void handle(const T &, State &gs)
{
    // All unhandled events will end up here
}
///
////////////////////////////////////

int main()
{
    State gs(800, 600, "Text echo");
    sf::Text logText{gs.font, "", FONT_SIZE};
    while (gs.window.isOpen()) // main loop
    {
        // event loop and handler through callbacks
        gs.window.handleEvents([&](const auto &event)

```

```

        { handle(event, gs); });

// Show log
gs.window.clear();
if (gs.log.size() > 24u) // Limit the log to the last 24 entries
    gs.log.erase(gs.log.begin(), gs.log.begin() +
static_cast<int>(gs.log.size() - 24u));
for (std::size_t i = 0; i < gs.log.size(); ++i)
{
    logText.setPosition({50.f, static_cast<float>(i * FONT_SIZE) + 50.f});
    logText.setString(gs.log[i]);
    gs.window.draw(logText);
}
gs.window.display();
}
}

```

Questo è il codice iniziale che ci viene fornito, abbiamo un text buffer che è un vettore di stringe, funge da “memoria” per registrare e conservare ciò che l’utente digita. Ogni elemento all’interno del vettore rappresenta una singola riga di testo. Nella funzione handle per **TextEntered** se premiamo invio (\n o \r) il programma aggiunge una nuova riga vuota al buffer usando `gs.log.emplace_back(“”)`; Se inseriamo un carattere normale questo viene concatenato alla fine dell’ultima stringa presente nel buffer `gs.log[last] += static_cast<char>(textEnter.unicode)` Nel loop del main il programma legge il buffer per mostrare il testo a schermo. Controlla anche che non superi le 24 righe.

A questo punto dichiariamo due variabili:

```

sf::IntRect text_view;
sf::Vector2i cursor_pos;

```

text_view è un rettangolo a coordinate intere. Mentre **cursors_pos** è semplicemente il cursore del mouse. Come viene spiegato dal testo quello che dobbiamo fare è **settare l’origine** di textview a {0, 0} , la sua **larghezza** è **window.getSize().x / FONT_WIDTH - 2** e infine la sua altezza è **window.getSize().y / FONT_SIZE - 2**. Il cursore viene semplicemente posizionato a {0 , 0} quindi il codice è:

```

text_view = { {0, 0}, {static_cast<int>(window.getSize().x / FONT_WIDTH - 2),
static_cast<int>(window.getSize().y / FONT_SIZE - 2)}};
cursors_pos = {0, 0};

```

Il punto 6 e 7 del testo dicono che dobbiamo togliere la logica dalla riga **gs.window.clear()** fino a **gs.window.display()** e di inserirle dentro a un metodo chiamato **doGraphics**. Il punto 7 ci chiede di cancellare le linee che cancellano le righe del testo (vogliamo conservarle tutte). Quindi il metodo è il seguente:

```
void doGraphics(State &gs) {
    sf::Text logText{gs.font, "", FONT_SIZE};

    gs.window.clear();
    for (std::size_t i = 0; i < gs.log.size(); ++i)
    {
        logText.setPosition({FONT_WIDTH, static_cast<float>(i * FONT_SIZE) +
FONT_SIZE});
        logText.setString(gs.log[i]);
        gs.window.draw(logText);
    }
}
```

Per il punto 8 invece dobbiamo mostrare il cursore e settarli un colore verde, quindi facciamo:

```
logText.setPosition({static_cast<float>((gs.cursor_pos.x - gs.text_view.position.x +
1) * FONT_WIDTH), static_cast<float>((gs.cursor_pos.y - gs.text_view.position.y
+ 1) * FONT_SIZE + 3)});
logText.setString("_");
logText.setFillColor(sf::Color::Green);
gs.window.draw(logText);
gs.window.display();
```

Il sistema di coordinate è simile al text view.

Il metodo `adjust view` serve per mantenere sempre il cursore nell'area visibile. Il blocco **`text_view.size`** converte i pixel dello schermo in “griglia di testo”.

dividendo la larghezza della finestra per la larghezza di un singolo carattere (`FONT_WIDTH`) si ottiene il numero di colonne. Il `-2` serve per creare un margine in modo che il testo non tocchi i bordi della finestra.

Il blocco **`std::min`** è lo scorrimento indietro. Se la posizione del cursore (`cursor_pos.x`) diventa minore (più a sinistra) della vista attuale (`text_view.position.x`) allora la vista si sposta a sinistra per agganciarsi al cursore.

Il blocco **`std::max`** è lo scorrimento in avanti. Se il cursore va oltre il limite destro visibile, la vista deve scorrere. La formula **`cursor_pos.x - text_view.size.x + 1`** calcola esattamente l'offset necessario affinché l'ultimo carattere a destra sia quello su cui si trova il cursore. il `+1` serve per l'indice 0.

```
void adjustView() {  
    text_view.size = {static_cast<int>(window.getSize().x / FONT_WIDTH - 2),  
                      static_cast<int>(window.getSize().y / FONT_SIZE - 2)};  
  
    text_view.position.x = std::min(text_view.position.x, cursor_pos.x);  
    text_view.position.y = std::min(text_view.position.y, cursor_pos.y);  
  
    text_view.position.x = std::max(text_view.position.x, cursor_pos.x -  
text_view.size.x + 1);  
    text_view.position.y = std::max(text_view.position.y, cursor_pos.y -  
text_view.size.y + 1);  
}
```

Per il punto 10 dobbiamo aggiungere il calcolo di quante righe stampare, dichiariamo `lines_to_print`:

```
unsigned lines_to_print = std::min(
    static_cast<unsigned>(gs.log.size() - gs.text_view.position.y),
    static_cast<unsigned>(gs.text_view.size.y)
);
```

Ci dobbiamo ricordare che **gs.log** contiene TUTTE le righe di testo, **gs.text_view.position.y** è la riga da cui parte la visualizzazione(scroll verticale) e **gs.text_view.size.y** sono quante righe stanno nello schermo, il min serve per non andare oltre il testo disponibile.

Modifichiamo il ciclo for in questo modo:

```
for (unsigned i = 0; i < lines_to_print; ++i)
```

Aggiungiamo un controllo sulla colonna visibile:

```
if(gs.text_view.position.x < gs.log[i + gs.text_view.position.y].size())
```

Controlla se la colonna da cui stiamo visualizzando il testo esiste. Dentro all'if mettiamo:

```
logText.setString(gs.log[i + gs.text_view.position.y].substr((size_t)gs.text_view.position.x,
    (size_t)gs.text_view.size.x));
```

Questa riga di codice serve a decidere quale parte della riga di testo visualizzare nella finestra, gestendo lo scroll orizzontale. `gs.log` contiene tutte le righe del testo, `i` è la riga relativa nella finestra mentre **text_view.position.y** è la riga da cui parte la visualizzazione e la loro somma ci dà la riga reale nel testo. La parte

.substr(start,length) serve a prendere solo una porzione della stringa, lo **start** sarebbe da quale carattere iniziare mentre il **length** quanti caratteri prendere.

Nel nostro caso **start** è **(size_t)gs.textview.position.x** e indica la colonna da cui iniziare a visualizzare. **Lenght** è **(size_t)gs.text_view.size.x** e indica quanti caratteri possono stare nella finestra.

Secondo laboratorio

{guardare il testo su GitHub}

Per il **primo punto** ci basta aggiungere una variabile(dentro a State):

```
int menubar = 50
```

Il canvas è implicitamente tutto il resto, quindi il primo punto finisce in questo modo. Per il **punto 3** dobbiamo solo aggiungere un array di 7 colori all'interno di State e scriviamo:

```
sf::Color colors[7] = {sf::Color::White, sf::Color::Red, sf::Color::Green,  
sf::Color::Blue, sf::Color::Cyan, sf::Color::Yellow, sf::Color::Magenta};
```

Anche se il testo parla di forme, le implementiamo nei punti successivi. Per il **punto 4** disegniamo i bottoni delle forme (il punto 4 estende la logica del punto 3):

```
void doGUI(State &gs)  
{  
    float button_size = gs.menubar;  
    // Setup del bottone  
    sf::RectangleShape button({button_size, button_size})  
    button.setFillColor(sf::Color::Transparent); // interno trasparente  
    button.setOutlineThickness(-3.f); // spessore negativo per bordo interno  
  
    // disegno dei 5 bottoni  
    for(int i = 0; i < 5; ++i)  
    {  
        button.setPosition(button_size * i, 0); // questo calcolo serve a  
        posizionare i bottoni uno di fianco all'altro  
        button.setOutlineColor(GRAY);  
        gs.window.draw(button)  
    }  
}
```

Ora implementiamo la logica dei 5 bottoni:

```
void doGUI(State &gs)  
{  
    float button_size = gs.menubar;  
    float shape_radius = button_size * 3.0 / 10.0; // approssimazione per farlo  
    stare entro i 22 pixel  
    // Setup del bottone
```

```

sf::RectangleShape button({button_size, button_size})
button.setFillColor(sf::Color::Transparent); // interno trasparente
button.setOutlineThickness(-3.f); // spessore negativo per bordo interno

// punto 5
sf::CircleShape shape(shape_radius);
shape.setOrigin({shape_radius, shape_radius}); // sposta il centro della forma

// disegno dei 5 bottoni
for(int i = 0; i < 5; ++i)
{
    button.setPosition(button_size * i, 0); // questo calcolo serve a
posizionare i bottoni uno di fianco all'altro
    button.setOutlineColor(GRAY);
    gs.window.draw(button)

    // posiziona, configura e disegna la forma (punto 5)
    shape.setPosition({static_cast<float>(button_size * (i + 0.5)), button_size
/ 2});

    // regola dei vertici (Punto 3 applicato qui)
    if (i > 0)
        shape.setPointCount(2 + i);
    else
        shape.setPointCount(30);

    gs.window.draw(shape);
}
}

```

Il testo chiede che le forme non siano in contatto col bordo del bottone. Siccome il bottone è largo 50 pixel e ha un bordo interno di 3 pixel, lo spazio utile al suo interno è: $50 - 3 \text{ (bordo sinistro)} - 3 \text{ (bordo destro)} = 44 \text{ pixel}$. Il raggio massimo prima di toccare i bordi è 22 pixel (la sua metà). Nel codice abbiamo usato la proporzione $3.0 / 10.0$ (il 30%, che fa 15 quindi < 22).

Spieghiamo questa formula: `shape.setPosition({static_cast<float>(button_size * (i + 0.5)), button_size / 2});`

La coordinata X è: **`button_size * (i + 0.5)`**, il prodotto posiziona esattamente all'inizio (bordo sinistro) del bottone corrente. Lo 0.5 serve per aggiungere la metà della larghezza del bottone, portando la X al centro orizzontale perfetto. I bottoni

partono dal bordo alto ($Y = 0$). Dividendo l'altezza del bottone per 2, si ottiene il centro verticale. Per il punto 6 dobbiamo aggiungere i colori ai bottoni:

```
// disegno colori menu
for(int i = 0; i < 7; ++i)
{
    // posiziona partendo dal bordo destro
    button.setPosition((gs.window.getSize().x + button_size * (i - 7), 0));
    button.setOutlineColor(GRAY);
    // pesca il colore dal nostro array nello State
    button.setFillColor(gs.colors[i]);

    gs.window.draw(button);
}
```

il `gs.window.getSize().x` prende la larghezza totale della finestra in quel momento, il **`button_size * (i - 7)`** fa un “passo indietro” dal bordo destro per fare spazio esattamente ai 7 bottoni. Al primo giro ($i = 0$) la posizione sarà $800 + 50 * (-7) = 450$. Così i 7 bottoni si incastrano perfettamente fino al margine destro. Per il **punto 7** dentro a State aggiungiamo due variabili di tipo unsigned (in quanto non possono essere negative):

```
unsigned active_shape = 0;
unsigned active_color = 0;
```

Dentro a **doGui** dobbiamo modificare la funzione in modo che disegni con un colore diverso il bordo dei bottoni corrispondenti alla forma e al colore attivi.

Quindi nel primo ciclo for aggiungiamo:

```
if (i == gs.active_shape)
    button.setOutlineColor(LIGHT_GRAY); // Selezionato
else
    button.setOutlineColor(DARK_GRAY);  // Non selezionato
```

Ora modifichiamo la handle per **MouseButtonPressed**:

```
void handle(const sf::Event::MouseButtonPressed &mouseBP, State &gs)
{
    sf::Vector2i pos = mouseBP.position;
    unsigned button_size = gs.menubar;

    // controlla se il click è nell'area alta dei menu (y <= 50 )
    if(pos.y <= gs.menubar) {
        // se è a sinistra (menu forme)
```

```

        if(pos.x <= 5 * button_size)
            gs.active_shape = pos.x / button_size;
        // se è a destra (menu colori)
        else if(pos.x >= gs.window.getSize().x - (7 * button_size))
            gs.active_color = 6 - (gs.window.getSize().x - pos.x) / button_size;
    }
}

```

Per il **punto 8** dobbiamo creare la struttura dati myShape:

```

struct myShape
{
    int type;
    int color;
    float size = 30.f;
    sf::Vector2f pos;
}

```

All'interno della struct **State**, dobbiamo aggiungere un vettore (**vector**) di tali struct per memorizzare le shape disegnate, che all'inizio sarà vuoto:

```
std::vector<myShape> shapes;
```

Dobbiamo aggiungere un blocco nell'else dell'if della menu bar del punto 7, in quanto dobbiamo verificare se l'evento avviene quando il cursore del mouse si trova nella canvas e proviene dal bottone sinistro.

```

if (but == sf::Mouse::Button::Left) // click sinistro
{
    myShape s;
    s.pos = {static_cast<float>(pos.x), static_cast<float>(pos.y)};
    s.type = gs.active_shape;
    s.color = gs.active_color;
    gs.shapes.emplace_back(s); // aggiunge la forma al vettore
}

```

Infine per il **punto 8** nella doGraphics dobbiamo cliccare sul contenuto di tale vettore e disegnare le shape nella canvas. Aggiungiamo questo codice:

```

for (int i = 0; i < gs.shapes.size(); i++)
{
    myShape s = gs.shapes[i];
    sf::CircleShape toDraw(s.size);
    toDraw.setOrigin({s.size, s.size}); // Centra il punto di disegno
    toDraw.setPosition(s.pos);
    toDraw.setFillColor(gs.colors[s.color]); // Assegna il colore giusto
}

```

```

        // Regola dei vertici vista nel Punto 3
        if (s.type > 0)
            toDraw.setPointCount(s.type + 2);

        gs.window.draw(toDraw);
    }

```

Il **punto 9** gestisce la selezione e lo spostamento delle forme. Aggiungiamo allo State due variabili per gestire la selezione e il movimento:

- `selected_shape` inizializzata a -1
- `mouse_pos` per ricordare dove si trovava il mouse al frame precedente

```

unsigned active_shape = 0;
unsigned active_color = 0;
unsigned select_shape = -1;
sf::Vector2i mouse_pos;

```

Nella funzione handle di **MouseButtonPressed**, dentro l'else della canvas, gestiamo il click destro:

```

else if (but == sf::Mouse::Button::Right) {
    gs.mouse_pos = pos;
    for (int i = 0; i < gs.shapes.size(); ++i) {
        if (dist(gs.shapes[i].pos, pos) < gs.shapes[i].size) {
            gs.selected_shape = i;
            break;
        }
    }
}

```

Nella funzione handle di **mouseMoved**, se abbiamo una forma selezionata, calcoliamo la differenza di pixel tra la posizione attuale del mouse e quella precedente, spostiamo la forma e aggiorniamo **mouse_pos**:

```

void handle(const sf::Event::MouseMove &mouseMoved, State &gs)
{
    if (gs.selected_shape != -1) {
        gs.shapes[gs.selected_shape].pos.x += mouseMoved.position.x -
gs.mouse_pos.x;
        gs.shapes[gs.selected_shape].pos.y += mouseMoved.position.y -
gs.mouse_pos.y;
        gs.mouse_pos = mouseMoved.position; // aggiorniamo la posizione
    }
}

```

Quando lasciamo il tasto destro, dobbiamo dire al programma di “sganciare” la forma:

```
void handle(const sf::Event::MouseButtonReleased &, State &gs)
{
    if (gs.selected_shape != -1)
        gs.selected_shape = -1;
}
```

Infine nel ciclo for dentro alla **doGraphics**, aggiungiamo un controllo per disegnare un bordo grigio chiaro solo se la forma che stiamo disegnando è quella attualmente selezionata:

```
if (i == gs.select_shape) {
    toDraw.setOutlineColor(LIGHT_GRAY);
    toDraw.setOutlineThickness(-3.f);
} else {
    toDraw.setOutlineThickness(0.f); // Nessun bordo per le altre
}
```

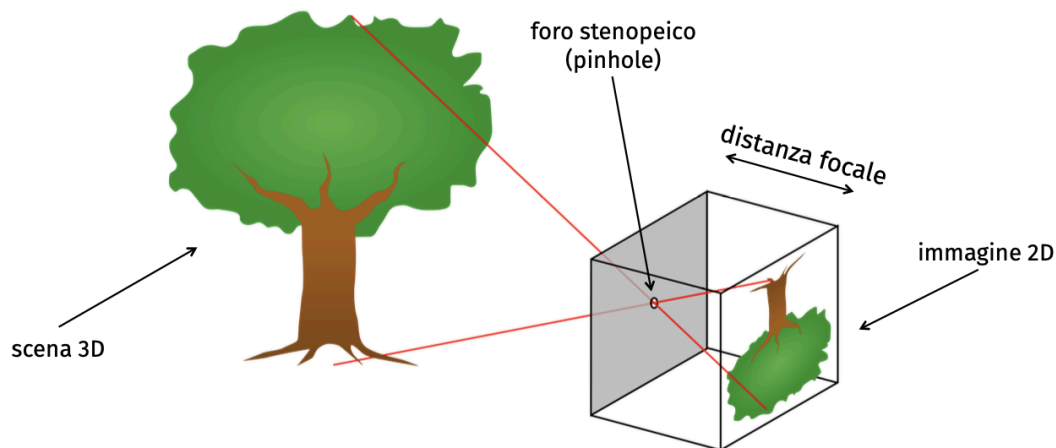
Per il **punto 10** facciamo:

```
void handle(const sf::Event::KeyPressed &keyPressed, State &gs)
{
    // se c'è una forma selezionata E il tasto premuto è Backspace
    if (gs.selected_shape != -1 && keyPressed.code==sf::Keyboard::Key::Backspace)
    {
        // rimuove la forma dal vettore calcolando la posizione esatta
        gs.shapes.erase(gs.shapes.begin() + gs.selected_shape);

        // resetta lo stato della selezione
        gs.selected_shape = -1;
    }
}
```

Computer grafica 3D

La camera oscura



114468 - 2025/26 - Fondamenti di Computer Grafica - Enrico Puppo

L'immagine illustra il modello della **Camera Oscura** (o telecamera *pinhole*). È il principio fisico di base usato in computer grafica per spiegare come un ambiente tridimensionale viene proiettato e trasformato in un'immagine **bidimensionale piatta** (come lo schermo del computer). La **scena 3D** è l'oggetto reale o virtuale che stiamo inquadrando. Il **pinhole** è il minuscolo buco da cui passa la luce. In grafica 3D questo rappresenta l'occhio dell'osservatore o la telecamera virtuale. La **distanza focale** non è altro che la distanza tra il foro e la parete di fondo (dove si trova l'immagine). Modificare questa distanza cambia il campo visivo agendo di fatto come lo zoom di una macchina fotografica. **L'immagine 2D** è il risultato della proiezione. Poichè i raggi di luce viaggiano in linea retta e si incrociano passando per il piccolo foro, l'immagine che si crea sulla parete di fondo risulta capovolta.

I passi fondamentali sono:

1. Creare la scena da visualizzare (modeling)
2. Definire l'osservatore (viewing)
3. Generare l'immagine della scena dal punto di vista dell'osservatore (rendering)

Costruzione della scena

Una scena è costituita da oggetti:

- **forma geometrica:** la sua struttura fisica in 3D. (è un cubo? una sfera? ecc.)
- **proprietà ottiche:** il suo materiale e il suo aspetto.

Come si creano e come si posizionano:

- **Modellazione geometrica:** è il processo con cui si “scolpisce” la forma geometrica dell’oggetto nel computer.
- **Trasformazioni geometriche:** una volta creato l’oggetto, lo dobbiamo mettere al posto giusto sul set. Le trasformazioni sono le operazioni matematiche che ti permettono di prendere un oggetto e per esempio spostarlo nello spazio (traslazione), ruotarlo e ingrandirlo (scalatura)

Per questo corso si limiteremo ai metodi più semplici per definire oggetti utili alla grafica raster:

- forme descritte da mesh di triangoli
- proprietà ottiche descritte da semplici parametri
- texture

Il “mattoncino” fondamentale della **grafica raster** è il triangolo. La superficie di ogni oggetto è costituita da un insieme di triangoli "appiccicati insieme": ogni oggetto di questo tipo si chiama **mesh geometrica**. La scena da visualizzare, a basso livello, è un insieme di triangoli che possono essere molto piccoli. I parametri che descrivono le proprietà ottiche sono associati ai triangoli o ai loro vertici .

Di norma, le mesh si creano con un’applicazione esterna (modellatore). L’applicazione grafica carica le mesh da file. Per costruire la scena dobbiamo istanziare e posizionare gli oggetti:

- trasformazioni geometriche 3D: traslazioni, rotazioni, scalature
- come in 2D ma più complesso



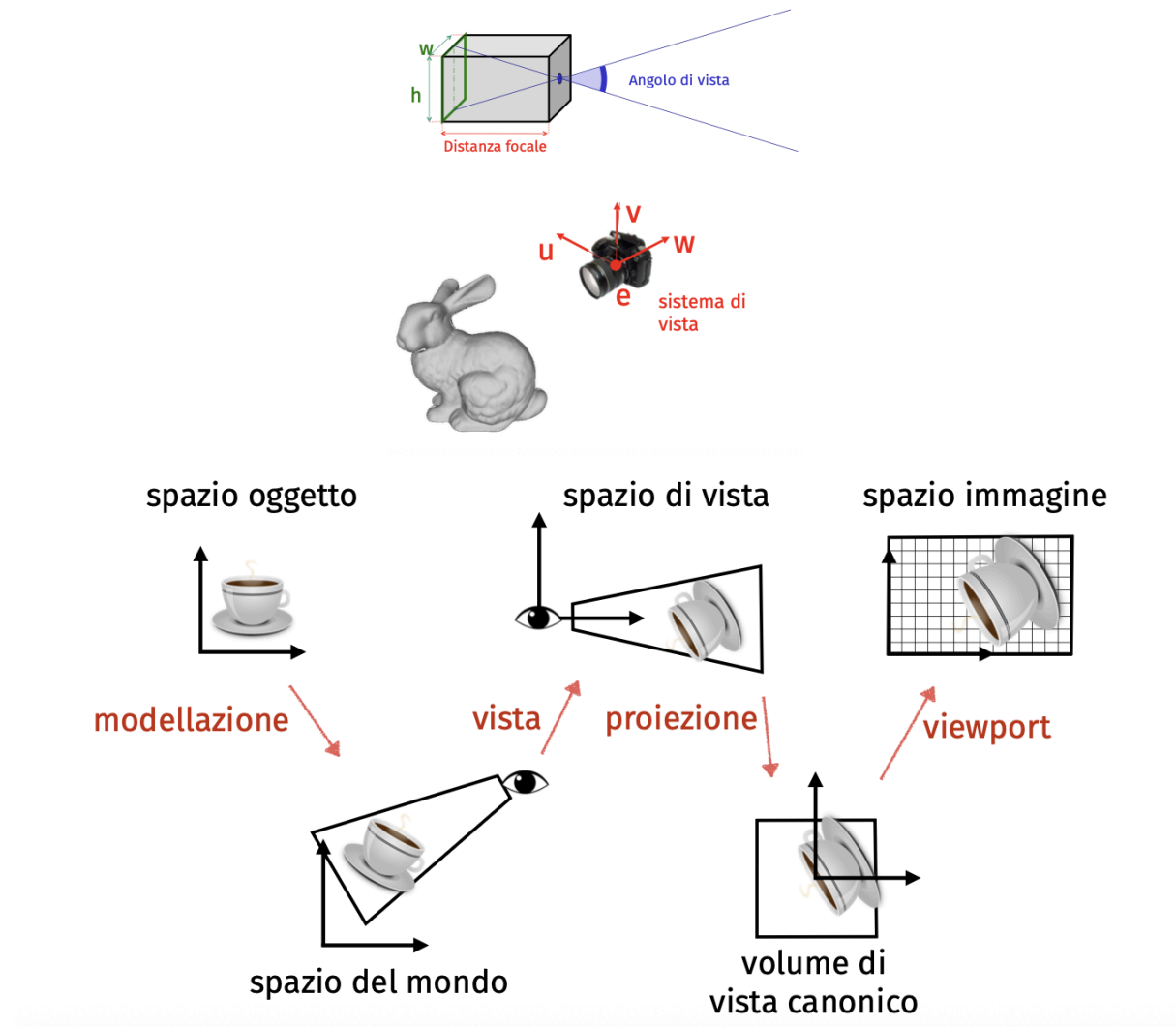
creano con un’applicazione esterna (modellatore). L’applicazione grafica carica le mesh da file. Per costruire la scena dobbiamo istanziare e posizionare gli oggetti:

Definizione dell’osservatore

Un osservatore è definito da:

- **parametri intrinseci:** definiscono la camera pinhole con cui l'osservatore guarda la scena
- **parametri estrinseci:** definiscono come l'osservatore posiziona la camera pinhole rispetto alla scena

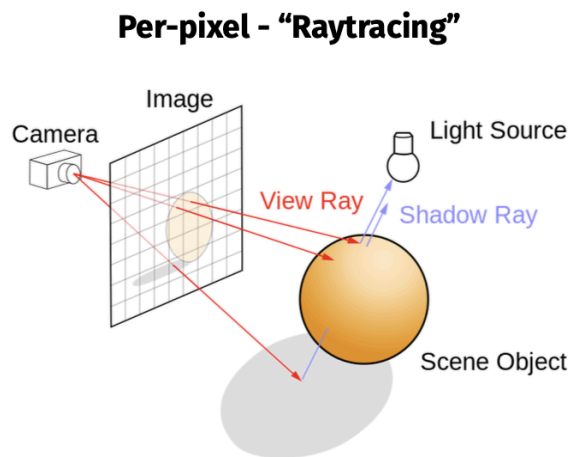
Anche questo porterà a definire trasformazioni geometriche.



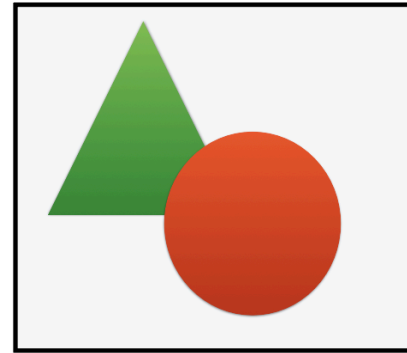
La tazza viene creata(modellata) nel suo sistema di coordinate isolato dove lei è al centro del suo piccolo universo. L'operazione di **modellazione** sposta,ruota e scala la tazza per inserirla nella scena globale. **Spazio del mondo:** ora la tazza è posizionata nel "set", con un unico sistema di coordinate(il mondo). **Spazio di vista:** il sistema di riferimento è ora allineato con la telecamera. Vediamo la tazza esattamente dalla prospettiva dell'osservatore. **La proiezione** applica le regole della prospettiva per iniziare a "schiacciare" il 3D verso il 2D. **Volume di vista canonico**: il cono visivo della telecamera viene deformato matematicamente in un cubo standard perfetto. Questo serve al computer per tagliare via velocemente ciò

che sta fuori dall'inquadratura. Il **viewport** mappa questo cubo matematico sulle dimensioni fisiche del nostro schermo in pixel (es. 1920 x 1080).

Paradigmi di Rendering

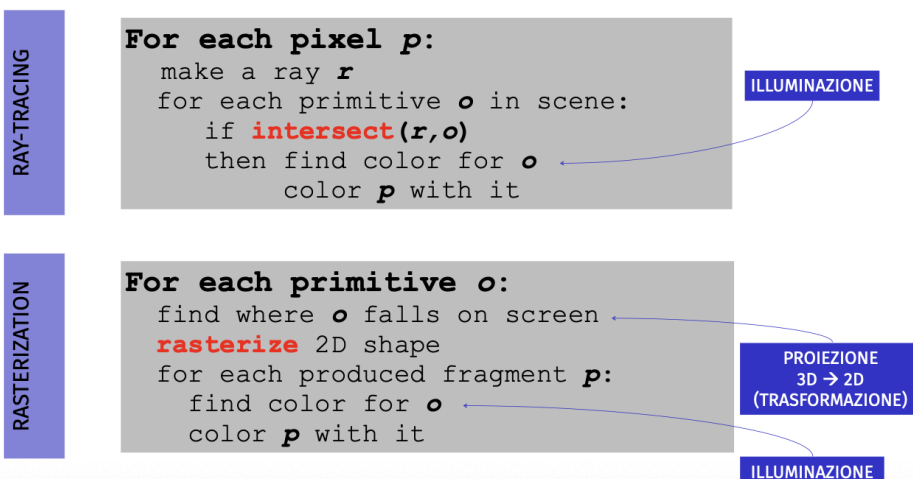


Per-oggetto - "Rasterizzazione"



Il computer si chiede “per ogni singolo pixel del mio schermo, che colore devo metterci?”. L'immagine di sinistra mostra come la **Camera** lancia un raggio visivo virtuale (view Ray), lo fa passare attraverso un pixel dello schermo e viaggia nella scena 3D finché non sbatte contro un oggetto (in questo caso la sfera). A quel punto, lancia un altro raggio (shadow Ray) verso la luce per capire se quel punto è in ombra o illuminato. Questo modello calcola fisicamente il comportamento della luce, è perfetto per ottenere fotorealismo estremo ma richiede una potenza di calcolo enorme. Mentre con la **rasterizzazione** il computer si chiede “per ogni oggetto 3D che ho nella scena, quali pixel dello schermo devo colorare?”. In breve **la rasterizzazione** è il processo matematico che decide quali quadretti (pixel) colorare per rappresentare meglio una determinata forma.

Algoritmi di Rendering



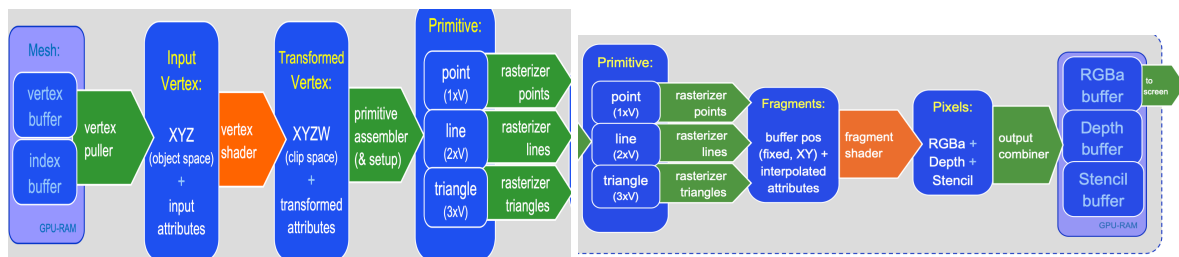
Rasterizzazione

Per ogni primitiva (es. il triangolo di prima) il processo si divide in fasi logiche:

- **Proiezione:** si proietta l'oggetto sull'immagine tramite trasformazioni geometriche e operazioni di clipping
- **Individuazione:** capire quali pixel sono contenuti nell'oggetto proiettato
- **Colorazione:** colorare i pixel secondo le proprietà dell'oggetto, applicando lighting e shading

Il sistema deve calcolare quali oggetti in primo piano nascondono quelli dietro e gestire l'eventuale presenza di materiali semi-trasparenti.

Pipeline della GPU



- **Vertex Puller (VP)** trasferisce i vertici dai buffer alla memoria video
- **Vertex Shader(VS)** include le trasformazioni geometriche
- **Primitive Assembler(PA)** prende i vertici trasformati e li raggruppa (a gruppi di 1,2 o 3) per generare le primitive geometriche vere e proprie
- **Rasterizer(RS)** produce i frammenti. Associa ad ogni frammento una posizione fissa nel framebuffer e i vari attributi interpolati
- **Fragment Shader(FS)** produce i valori dei pixel incluso il colore
- **Output Combiner (OC)** stadio finale che applica test e operazioni aggiuntive

NOTA: vertex shader e fragment shader sono programmabili in GLSL

Ray Tracing

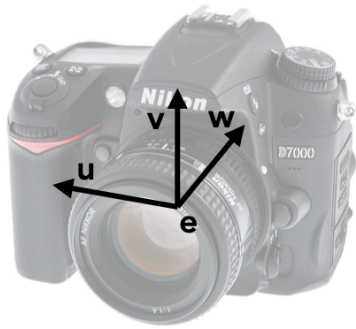
Schema base:

- Generazione dei raggi (uno per pixel)
- Intersezione del raggio con gli oggetti della scena

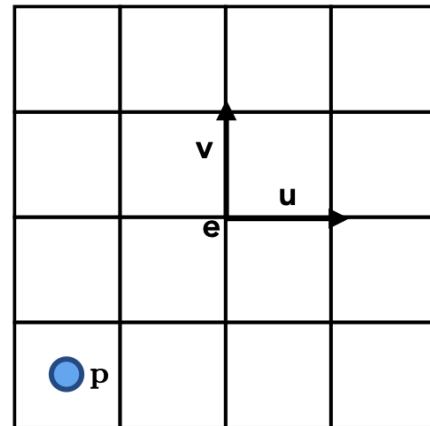
- Shading(calcolo del colore per pixel)

Calcolo dei raggi

1. Calcolo dei raggi



e origine del sistema di riferimento
p centro del pixel



$$\mathbf{p} = \mathbf{e} + u\mathbf{u} + v\mathbf{v} + w\mathbf{w}$$

L'osservatore(la camera) è definito da un'origine $\mathbf{e}$ e da un insieme di vettori $\mathbf{u}, \mathbf{v}, \mathbf{w}$. Il centro del pixel da colore si chiama $\mathbf{p}$. Il raggio assegnato al pixel $\mathbf{p}$ ha come origine il punto $\mathbf{e}$ e come direzione il vettore $\mathbf{p} - \mathbf{e}$. La percezione della profondità è data dalla distanza del piano immagine rispetto all'origine $\mathbf{e}$.

Intersezione

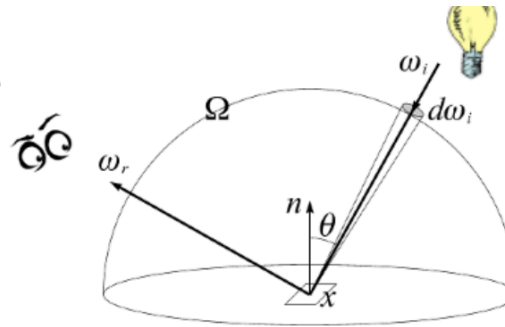
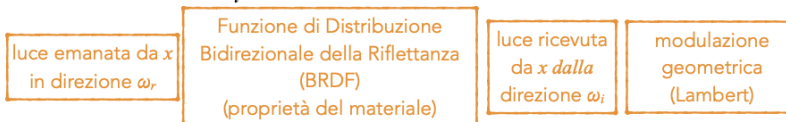
Scopo: calcolare le intersezioni di un raggio con gli oggetti presenti nella scena. Capire quale delle intersezioni è la più vicina all'osservatore. Operazione costosa, ci sono molti algoritmi specializzati secondo le primitive geometriche da intersecare.

Shading

- Modellare accuratamente l'interazione tra luce e superficie degli oggetti è complesso e costoso da calcolare

- Equazione della radianza:

$$L_r(x, \vec{\omega}_r) = \int_{\vec{\omega}_i \in \Omega} f_r(x, \vec{\omega}_i, \vec{\omega}_r) L_i(x, \vec{\omega}_i) (\vec{\omega}_i \cdot \vec{n}) d\vec{\omega}_i$$



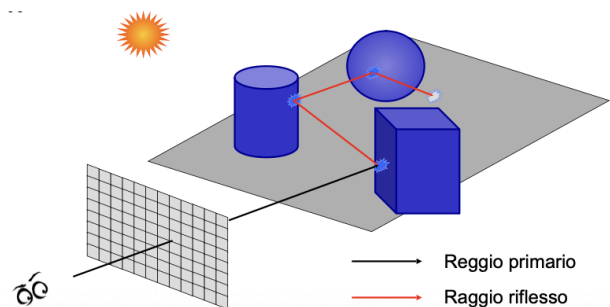
Il modello fisico è troppo complicato da calcolare e richiede troppe informazioni. Spesso si usa un'approssimazione semplice ed efficiente, che considera vari tipi di superficie:

- Diffuse o Lambertiane (opache)
- Speculari (lucide)
- Semi-trasparenti (rifrangenti)

NOTA: ogni tipo di superficie riflette la luce in modo diverso.

Propagazione per riflessione

Il raggio può rimbalzare diverse volte. A ogni rimbalzo raccoglie luce riflessa dell'oggetto colpito. Secondo il tipo di materiale, ci possono essere diversi raggi riflessi in diverse direzioni.



Grafica 3D raster - Mesh e trasformazioni

I passi fondamentali sono:

1. Creare la scena da visualizzare (modeling)
2. Definire l'osservatore (viewing)

3. Generare l'immagine della scena dal punto di vista dell'osservatore(rendering)

Gli “ingredienti” sono:

1. Mesh di triangoli
2. Trasformazioni geometriche per disporle nella scena

Mesh Poligonali

In computer grafica gli oggetti sono costruiti unendo tanti piccoli poligoni, chiamati **Mesh**.

- Una mesh è composta di *celle* di varie dimensioni

- 0-cella: *vertice* v_i

- 1-cella: *spigolo* e

- 2-cella: *faccia*



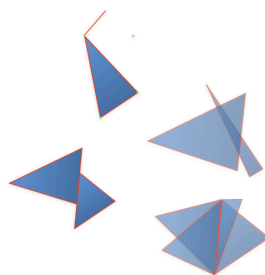
I **vertici** sono punti che definiscono le posizioni nello spazio.

Gli **spigoli** sono segmenti di retta che collegano coppie di vertici. Le **facce** sono regioni poligonali contornate da spigoli. L'intersezione di due spigoli (facce) è vuota o coincide con un insieme di vertici (spigoli).

Struttura delle mesh

Le configurazioni che NON vanno bene sono:

- spigoli “penzolanti” e vertici isolati
- facce che si intersecano
- adiacenza non-conformale
- spigoli con più di due facce incidenti



Tutte le facce sono **triangolari**. Le celle si chiamano **simplessi**: sono combinazioni convesse dei loro vertici.

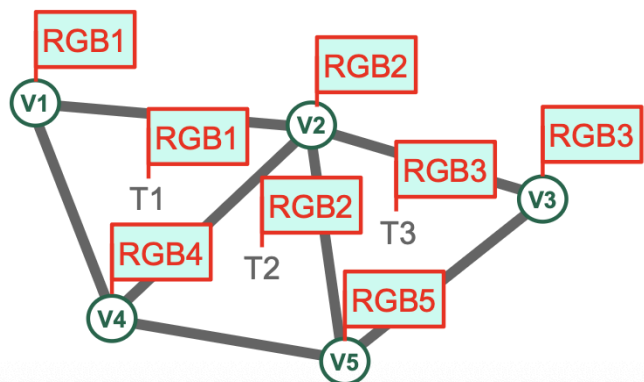
$$P = \lambda_0 V_0 + \lambda_1 V_1 + \lambda_2 V_2 \quad \lambda_i \in [0,1] \quad \lambda_0 + \lambda_1 + \lambda_2 = 1$$

Posizione dei vertici + struttura combinatoria caratterizzano la forma dell'oggetto definito da una mesh.

NOTA: La struttura combinatoria è semplicemente l'**elenco delle connessioni**. Ti dice *quali* vertici si uniscono tra loro per formare l'intera rete di triangoli.

Mesh attributi

- Valori che variano sulla superficie
 - definiti per vertice, interpolati per faccia, oppure
 - definiti per faccia



9186 - 2023/24 - Computer Graphics - Enrico Puppo

Ogni elemento della mesh può trasportare **attributi**, ovvero dati necessari per il rendering:

- **Colore(RGB)**
- **Vettore normale:** fondamentale per lo shading
- **Coordinate di texture**
- **Ossa(rigging):** per gestire animazioni

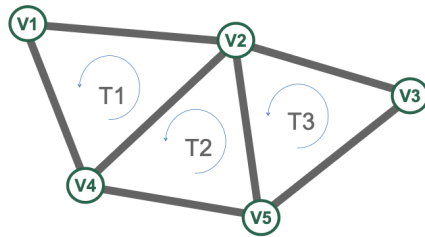
Strutture dati di base

Polygon soup = non è da usare, in quanto inefficiente. Questo perché si crea una lista di vertici contando anche le ripetizioni, quindi spreca memoria inutilmente.

Struttura indicizzata = Questa struttura risolve lo spreco di memoria separando i dati in due elenchi distinti

- **Geometria:** È un singolo elenco senza ripetizioni di tutti i vertici, dove per ognuno si salva la posizione e i vari attributi una sola volta
- **Connettività:** È un elenco di triangoli. Invece di riscrivere le coordinate, per ogni triangolo si salvano solo tre indici numerici che puntano alla posizione dei vertici corrispondenti all'interno del primo elenco

Struttura indicizzata (e.g., OFF files)



vert	X	Y	Z	R	G	B
V1	x1	y1	z1	r1	g1	b1
V2	x2	y2	z2	r2	g2	b2
V3	x3	y3	z3	r3	g3	b3
V4	x4	y4	z4	r4	g4	b4
V5	x5	y5	z5	r5	g5	b5

GEOMETRIA + ATTRIBUTI

Tri:	Wedge 1:	Wedge 2:	Wedge 3:
T1	V4	V2	V1
T2	V4	V5	V2
T3	V5	V3	V2

CONNETTIVITÀ

L'orientamento delle facce sono:

- **antiorario:** fronte/esterno
- **orario:** retro/interno

Trasformazioni geometriche

Ogni trasformazione lineare 2D si può rappresentare con una matrice 2x2

$$\begin{pmatrix} x' \\ y' \end{pmatrix} = \begin{pmatrix} a & b \\ c & d \end{pmatrix} \cdot \begin{pmatrix} x \\ y \end{pmatrix}$$

La **concatenazione di trasformazioni lineari** corrisponde a una moltiplicazione di matrici:

$$L_2(L_1(\mathbf{x})) = \mathbf{L}_2 \mathbf{L}_1 \mathbf{x}$$

Le trasformazioni lineari sono le più comuni in computer grafica.

Le operazioni standard sono:

- Scalatura
- Rotazione
- Traslazione

La traslazione non è una trasformazione lineare, ma è affine. L'origine non è un punto fisso. **Trasformazione affine** = trasformazione lineare + traslazione
Possiamo rappresentare le trasformazioni affini mediante matrici? sarebbe utile farlo per ottimizzare i calcoli

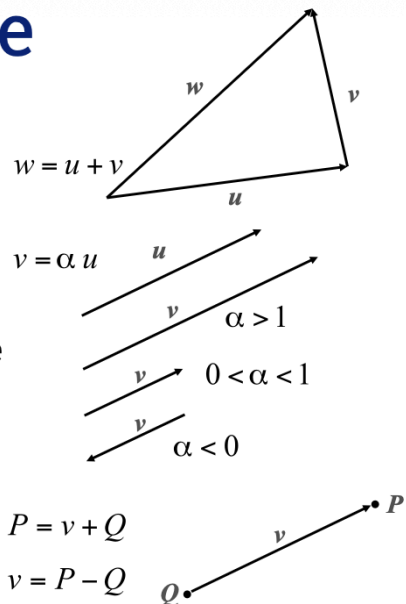
Algebra affine

I costituenti dell'algebra affine appartengono a tre classi:

- I **Punti** definiscono oggetti (posizioni nello spazio)
- I **Vettori** definiscono spostamenti (i vettori non hanno posizione nello spazio)
- Gli **Scalari** definiscono quantità

Algebra affine

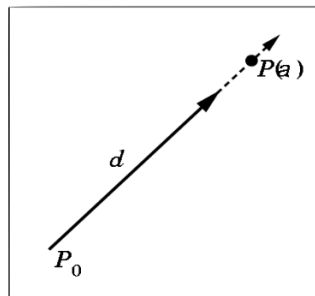
- Operazioni:
 - Aritmetica solita tra scalari
 - Somma vettore + vettore dà un altro vettore
 - Prodotto scalare $\cdot$ vettore cambia la norma (e il verso, se lo scalare è negativo) del vettore
 - Somma punto + vettore (spostamento) dà un altro punto
 - Differenza punto - punto dà un vettore



Retta nello spazio affine

- Rappresentazione parametrica:

$$P(\alpha) = P_0 + \alpha d$$



- Dati un punto P_0 e un vettore d , i punti di una retta parallela a d che passa per P_0 si ottengono facendo variare $\alpha \in (-\infty, +\infty)$

La retta nello spazio affine non è altro che un **insieme di punti** partendo da un punto base P_0 e spostandosi lungo la direzione di un vettore d .

La Retta nello Spazio Affine

In algebra affine, non usiamo la classica equazione $y = mx + q$. Una retta si definisce partendo da un **punto** e da un **vettore** (che indica la direzione). [PDF + 2](#)

- **Punto P_0** : La posizione di partenza sulla retta. [PDF + 1](#)
- **Vettore d** : La direzione in cui si muove la retta. [PDF](#)
- **Parametro α (alfa)**: Uno scalare (un numero) che ci dice "di quanto" ci spostiamo lungo la direzione d partendo da P_0 . [PDF](#)

La Formula Parametrica:

$$P(\alpha) = P_0 + \alpha d$$

Significato: Per trovare un punto P qualsiasi sulla retta, parti dal punto P_0 e spostati lungo il vettore d moltiplicato per il valore di α . Facendo variare α da $-\infty$ a $+\infty$, tracci tutta la retta infinita. (Nota che questa operazione, Punto + Vettore = Punto, è perfettamente lecita nell'algebra affine). [PDF + 3](#)

1. Ray Tracing (Simulazione della luce)

Nel ray tracing, ogni singolo raggio visivo sparato dalla telecamera è una retta affine descritta dall'equazione $P(\alpha) = P_0 + \alpha d$. L'algoritmo fa avanzare il parametro α per calcolare l'esatto punto di intersezione tra il raggio e i poligoni della scena. [PDF](#)

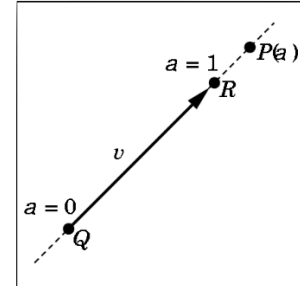
Somma affine

- Nell'algebra affine non hanno senso né la somma tra punti né il prodotto di uno scalare per un punto
- Ma esiste la *somma affine*:

$$v = R - Q$$

$$P = Q + \alpha v$$

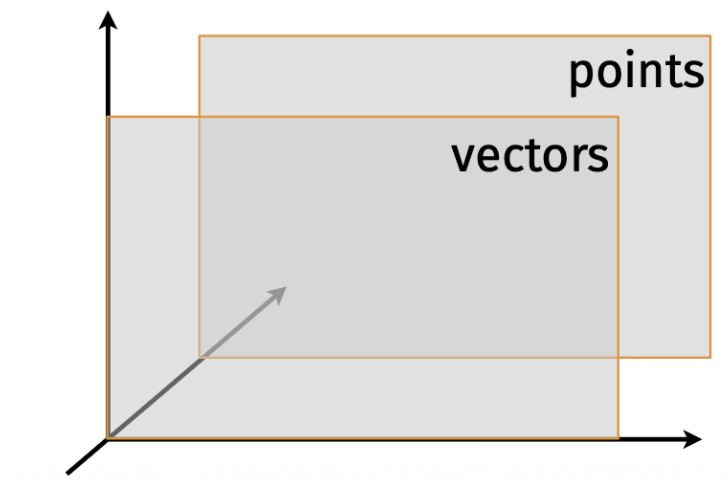
$$P = Q + \alpha(R - Q) = \alpha R + (1 - \alpha)Q$$



- La somma affine ci consente di interpolare (e eventualmente estrapolare) i punti P e Q

Spazio affine

Interpretazione geometrica: due piani paralleli in R^3 . I vettori stanno sul piano per l'origine. I punti stanno sul piano parallelo, con terza coordinata = 1.



Coordinate omogenee

Aggiungiamo una terza coordinata (coordinata w).

- punto 2D = $(x, y, 1)^T$
- vettore 2D = $(x, y, 0)^T$

Ora possiamo definire anche la traslazione attraverso una matrice:

$$\begin{pmatrix} x' \\ y' \\ w' \end{pmatrix} = \begin{pmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \\ 0 & 0 & 1 \end{pmatrix} \cdot \begin{pmatrix} x \\ y \\ 1 \end{pmatrix} = \begin{pmatrix} x + t_x \\ y + t_y \\ 1 \end{pmatrix}$$

Operazioni valide se la coordinata w vale 1 o 0:

- vettore + vettore = vettore
- punto - punto = vettore
- punto + vettore = punto
- punto + punto è indefinito (ci darebbe $w = 2$)
- a punto + $(1 - a)$ punto = punto
- Trasformazione affine = trasformazione lineare + traslazione

$$\begin{pmatrix} x' \\ y' \end{pmatrix} = \begin{pmatrix} a & b \\ c & d \end{pmatrix} \cdot \begin{pmatrix} x \\ y \end{pmatrix} + \begin{pmatrix} t_x \\ t_y \end{pmatrix}$$

- In coordinate omogenee:

$$\begin{pmatrix} x' \\ y' \\ 1 \end{pmatrix} = \begin{pmatrix} a & b & t_x \\ c & d & t_y \\ 0 & 0 & 1 \end{pmatrix} \cdot \begin{pmatrix} x \\ y \\ 1 \end{pmatrix}$$

Una cosa molto importante è che la moltiplicazioni di matrici non è commutativa. Infatti ad esempio:

Concatenazione di trasformazioni

- La moltiplicazione di matrici non è commutativa!
- Prima la rotazione, poi la traslazione:



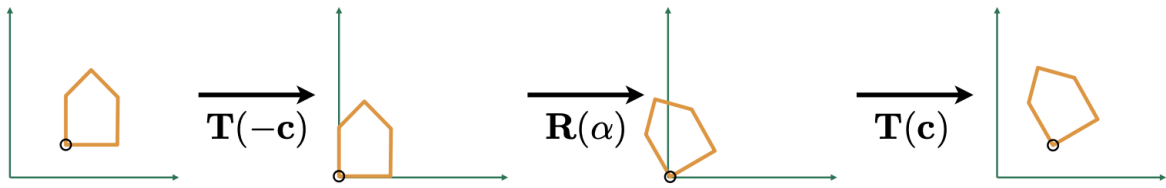
- Prima la traslazione, poi la rotazione:



Rotazione 2D

Come si ruota attorno a un punto c ?

Prima si trasla c nell'origine, poi si ruota e infine si effettua la traslazione opposta.



Rotazione 3D attorno all'origine

A cosa serve: permette di cambiare l'orientamento di un oggetto nello spazio.

Qualsiasi rotazione complessa in 3D può essere **scomposta e calcolata** come una sequenza di **tre rotazioni più semplici** attorno ai singoli assi X,Y,Z.

Nelle matrici di rotazione **i numeri non sono casuali**. Le righe/colonne rappresentano esattamente la nuova direzione dei tre assi coordinati dopo aver effettuato la rotazione.

Angoli di Eulero

È il metodo classico per definire l'orientamento di un oggetto tramite tre angoli in sequenza: **Yaw**(destra/sinistra sull'asse verticale), **Pitch**(su/giù asse orizzontale) e **Roll**(inclinazione sull'asse longitudinale). Calcolare la giusta combinazione di questi tre angoli per arrivare a una posizione arbitraria è matematicamente complicato.

Alternativa: al posto di fare tre rotazioni separate, il **teorema di Eulero** ci dice che qualsiasi orientamento 3D può essere raggiunto con un singolo movimento: basta trovare l'asse di rotazione corretto e ruotare l'oggetto di un solo angolo θ attorno ad esso.

Cambio di sistema di riferimento

Serve per tradurre le coordinate di un punto dal suo “sistema locale” al “sistema globale o mondo”. In sostanza è il processo matematico che permette di “tradurre” le coordinate di un punto o di un vettore da un sistema di assi a un altro.

Come funziona: si usa una **matrice fissa M** che funge da “dizionario” tra i due sistemi, convertendo istantaneamente i punti dall'uno all'altro.

Trasformazioni di Modellazione

Per piazzare correttamente un oggetto nella scena globale, c'è un ordine canonico da rispettare per evitare che gli oggetti orbitano a caso:

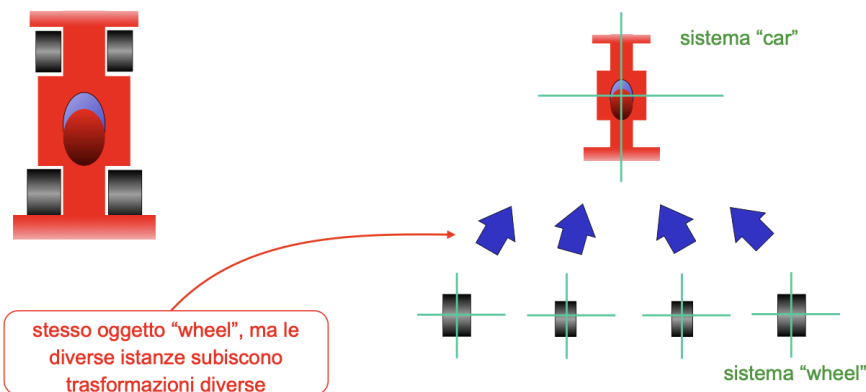
1. **Scalatura(S):** per dargli le dimensioni giuste
2. **Rotazione(R)**
3. **Traslazione(T)**

Tutto questo si “Impacchetta” moltiplicando le tre matrici $M = T R S$ per ottenere un'unica matrice di trasformazione.

Grafo di scena

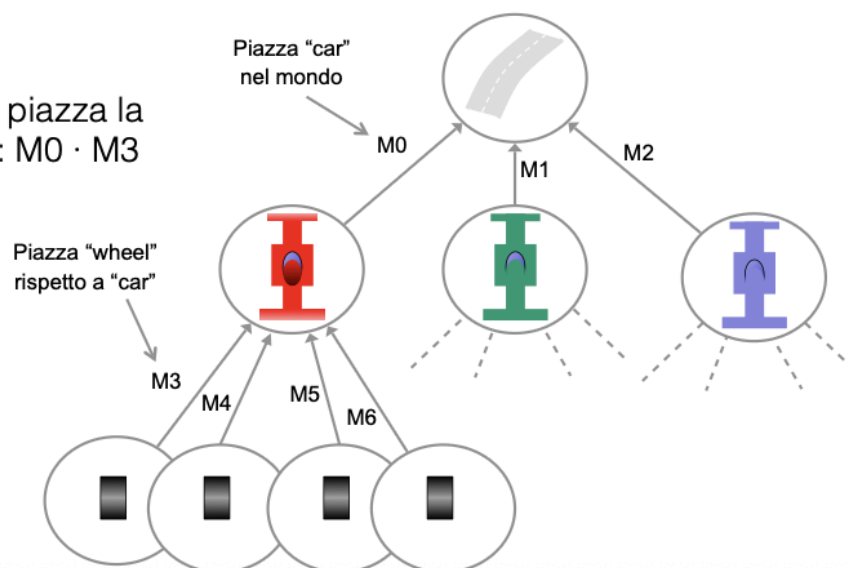
Cos'è: È una struttura dati ad albero (gerarchica) per gestire oggetti composti da più parti, come un'auto o un braccio robotico.

A cosa serve: Permette di "ereditare" le trasformazioni. Invece di spostare singolarmente quattro ruote e un telaio, raggruppi le ruote come "figli" dell'auto ("padre"). Se trasli o ruoti l'auto nel mondo, le ruote si spostano automaticamente assieme ad essa. Ogni "figlio" ha solo una piccola matrice di trasformazione relativa al proprio padre.



Grafo di scena

Matrice che piazza la prima ruota: $M0 \cdot M3$



Telecamera 3D

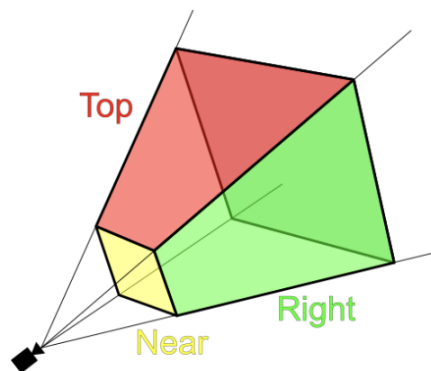
Per inquadrare una scena 3D, il computer ha bisogno di simulare una "macchina fotografica" (o telecamera virtuale). Questa si definisce con due tipi di parametri:

- **Parametri Estrinseci** (Dove sono e dove guardo): Riguardano la posizione fisica della telecamera nel mondo e il suo orientamento. Si definiscono con un punto di vista (l'occhio, e) e tre vettori direzionali (la direzione in cui guardi w , la linea verticale v e quella orizzontale u).
- **Parametri Intrinseci** (Che lente uso): Definiscono le caratteristiche ottiche dell'inquadratura. Usa la distanza focale o , più intuitivamente, il Field of View (FoV - Campo Visivo).

Volume di vista

L'insieme dei parametri **Intrinseci** ed **estrinseci** crea una forma geometrica solida (un tronco di piramide) che definisce **la porzione di mondo inquadrata**.

La forma dipende dai parametri **intrinseci (top e right) + piani di clipping**. Il clipping (near e far) decidono dove inizia e dove finisce la vista in profondità. Senza questi la vista sarebbe infinita. **La posizione e l'orientamento** dipendono dai parametri estrinseci.



Paradigmi di interazione

Descrivono come navighiamo nella scena 3D.

Come facciamo a far muovere la telecamera? ci sono due approcci:

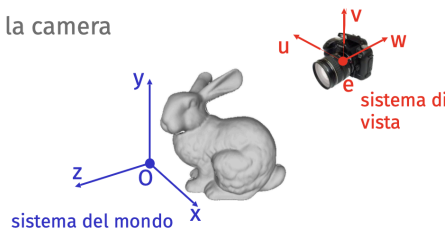
- **World in hand:** la telecamera sta ferma e tu ruoti il mondo davanti a lei
- **Camera in hand:** il mondo sta ferma e la telecamera ci cammina dentro (videogioco in prima persona)

Ci sono sempre due sistemi di coordinate in gioco:

1. **Sistema del mondo:** è l'universo fisso. È la stanza in cui piazzati gli oggetti (come il coniglio nella foto).
2. **Sistema di vista:** è il sistema incollata alla telecamera

Paradigmi di interazione

- In entrambi i casi si tratta di definire la relazione tra due sistemi di riferimento:
- Sistema del mondo, in cui viene modellata la scena
- Sistema di vista, solidale con la camera



Trasformazione di vista

Definizione: Passare dallo **spazio del mondo** allo **spazio di vista** richiede una trasformazione che è **esattamente l'inversa** di quella che piazzerebbe la telecamera nel mondo. **La vista** viene di solito definita da un punto in cui si trova l'occhio (**P_{pov}**), il punto che sta guardando (**P_{target}**) e il vettore che indica dove si trova l'alto (**V_{up}**).

{ questa definizione corrisponde alla matrice di vista, se all'orale viene chiesto cosa è la matrice di vista si può rispondere come sopra oppure in modo più diretto:

La matrice di vista è la trasformazione geometrica che permette di passare dalle coordinate espresse nello **spazio del mondo** (dove è modellata l'intera scena) alle coordinate nello **spazio di vista** (un sistema di riferimento solidale con la telecamera) . }

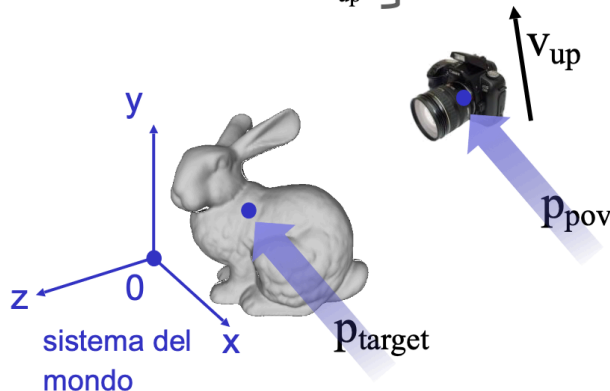
Input:

- 1) punto di vista:
- 2) punto target:
- 3) vettore "su":

p_{pov}
 p_{target}
 v_{up}

punti e vettori nel sistema del mondo!

Sufficiente a calcolare tutti i parametri estrinseci



In pratica: all'interno della **scheda video**, la telecamera è matematicamente fissa al centro dell'universo. Se in un videogioco il personaggio fa un passo avanti, la matematica non sposta la telecamera in avanti, ma prende l'intero universo (il mondo 3D) e lo sposta di un passo indietro. Per questo si calcola la **matrice "inversa"**. Se monti la telecamera su un'auto in movimento, la **matrice di vista** è l'inversa della **matrice di modellazione** dell'auto stessa.

Camera orbitante

È un sistema usato per ispezionare un singolo oggetto girandoci attorno. Invece di usare X,Y,Z la posizione della telecamera è definita da **coordinate sferiche**: **theta**(elevazione), **phi**(azimuth, ovvero rotazione orizzontale della telecamera) e **rho**(distanza dell'oggetto).

In pratica:

Per inquadrare l'oggetto (posizionare il mondo), il sistema prima trasla il mondo all'indietro della distanza **rho**, e poi lo ruota degli angoli **theta** e **phi**.

Per posizionare la camera nel mondo: Si arretra di **rho** lungo l'asse Z, si ruota di **-theta** lungo l'asse X e di **phi** lungo l'asse Y.

Object Viewer

L'object viewer è l'implementazione pratica del paradigma di interazione world-in-hand.

In questo approccio, la telecamera è concettualmente fissa e l'utente manipola la scena come se avesse l'oggetto tra le mani, mappando i movimenti del mouse sui

parametri di vista(ovvero P_{pov} , P_{target} e V_{up}). Le **operazioni fondamentali** sono tre:

1. **dolly** che modifica la distanza p tra l'occhio e l'oggetto con la rotella del mouse.
2. **il pan** ovvero spostamento trasversale. In pratica, si sposta la scena parallelamente al piano dello schermo.
3. **orbit/rotazione**: rotazione attorno all'oggetto mappata in base allo spostamento del mouse sugli assi X e Y dello schermo.

Quando guardi un modello 3D e lo giri un po', poi ti fermi, e poi lo giri ancora, l'oggetto non torna alla posizione di partenza. Il computer deve "**ricordare**" la posizione corrente (salvandola in una matrice chiamata V) e **accumulare** le nuove trasformazioni passo dopo passo. Visto che **in matematica le trasformazioni si leggono da destra a sinistra**, la nuova mossa che fai col mouse viene sempre moltiplicata a *sinistra* della matrice della posizione attuale (matrice V).

Se vuoi solo **traslare l'oggetto** (pan o dolly), per il computer è "un gioco da ragazzi". Prende la matrice attuale V , calcola la nuova piccola traslazione data dal tuo mouse, e le moltiplica. L'oggetto si sposta senza problemi.

Per **la rotazione** è più complicato: Se tu dici semplicemente al computer "ruota l'oggetto", lui lo ruoterà attorno all'origine della telecamera (il tuo occhio). Essendo l'oggetto distante da te, invece di girare su se stesso, farebbe un'orbita enorme uscendo fuori dallo schermo. Per farlo ruotare su se stesso, il computer deve fare un trucchetto a 4 passi:

1. Prende l'oggetto dalla sua posizione attuale
2. lo tira verso di se, mettendolo esattamente sull'origine T^{-1}
3. applica la rotazione data dal mouse R
4. Lo "spinge" di nuovo indietro alla distanza a cui si trovava T

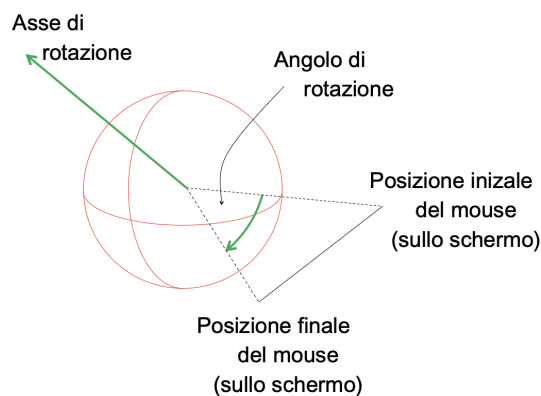
Il problema dell'ordine: Quando muovi il mouse in diagonale, stai chiedendo al computer di ruotare l'oggetto su due assi contemporaneamente (orizzontale e verticale). Ma c'è un **limite matematico**: la moltiplicazione tra matrici **non è commutativa**. Significa che applicare prima la rotazione orizzontale e poi verticale dà un risultato visivo diverso rispetto a fare prima verticale e poi orizzontale.

- Se muovi il mouse piano l'errore è invisibile
- Se fai un movimento veloce e lungo col mouse l'oggetto assumerà posizioni stranissime

La soluzione definitiva è la trackball.

Trackball

L'oggetto viene idealmente racchiuso in una sfera invisibile. Il sistema calcola dove i raggi, che partono dalle posizioni successive del mouse sullo schermo, intersecano questa sfera virtuale. Muovendo il mouse dalla posizione iniziale (P_0) alla finale (P_1), si ricavano esplicitamente un **asse di rotazione** e un **angolo**. **Combiniamo con la rotazione precedente.**



Il computer al posto di scomporre il movimento in X e Y calcola un'unica rotazione perfetta. Come fa? .

- **Trova l'Asse (v):** Il computer traccia due vettori che partono dal centro della sfera verso i punti P_0 e P_1 . Facendo un'operazione matematica chiamata *prodotto vettoriale* (indicato con $\times$) tra questi due, ottiene un nuovo vettore perpendicolare. Quello è l'esatto **asse di rotazione** (o perno) su cui l'oggetto dovrà girare ² .
- **Trova l'Angolo (α):** Calcola semplicemente la distanza tra il punto iniziale e quello finale sullo schermo. Più il movimento del mouse è stato lungo, più grande sarà l'angolo di rotazione calcolato ² .

Una volta calcolati l'asse v e l'angolo α , non puoi semplicemente moltiplicarli alla matrice di vista corrente V perché essa contiene già delle traslazioni (l'oggetto è a una distanza d dall'osservatore). Devi usare la tecnica del "sandwich": ⓘ+1

- **Ritorno all'origine:** Applichi una traslazione $T(d)$ per portare virtualmente il centro di rotazione nell'origine del sistema di vista. ⓘ+2
- **Rotazione:** Applichi la rotazione R' che hai appena calcolato con la trackball. ⓘ+1
- **Riposizionamento:** Applichi $T(-d)$ per riportare l'oggetto nella sua posizione originale davanti alla camera. ⓘ+1
- **Formula finale:** $V = T(-d) \cdot R' \cdot T(d) \cdot V$. ⓘ

Ancor meglio:

- Memorizziamo lo stato della trackball:
 - centro
 - raggio
 - rotazione corrente
- Codifichiamo le rotazioni mediante *quaternioni*
- Non vediamo la matematica dei quaternioni, ma:
 - una rotazione attorno a un asse si esprime con un quaternione
 - il prodotto di due quaternioni fornisce la rotazione composta
 - le librerie forniscono le conversioni tra rappresentazioni asse-angolo e quaternioni

Da orale:



Introduzione al Problema

All'orale puoi iniziare spiegando che, quando vogliamo ispezionare un oggetto 3D, usare le semplici rotazioni orizzontali e verticali legate al mouse porta a risultati poco intuitivi se i movimenti diventano ampi. La soluzione più elegante e naturale a questo problema è l'interfaccia a "Trackball". PDF + 2



La Sfera Virtuale

L'idea concettuale è semplice: immaginiamo di racchiudere l'oggetto 3D all'interno di una sfera virtuale. Quando l'utente clicca e trascina il mouse, il sistema prende la posizione iniziale e quella finale del cursore sullo schermo e "proietta" questi due punti direttamente sulla superficie della nostra sfera 3D. PDF + 2



Il Movimento

Avendo ora questi due punti sulla sfera, il sistema calcola geometricamente un asse di rotazione e un angolo. In parole povere, questo ci dice esattamente in che direzione e con quanta forza l'utente ha "fatto ruotare" la sfera con la mano. Questa nuova rotazione viene poi combinata con la rotazione che l'oggetto aveva in precedenza. PDF + 3



L'Ottimizzazione con i Quaternioni

Per concludere il discorso in modo brillante, puoi aggiungere che lo stato della trackball non viene gestito con le classiche matrici, ma è caldamente consigliato l'uso dei quaternioni. I quaternioni sono ideali perché ci permettono di esprimere una rotazione attorno a un asse in modo molto più stabile, e moltiplicandoli tra loro otteniamo la rotazione composta in maniera fluida, rapida e senza difetti visivi. PDF + 2

Trasformazioni di proiezioni

È definita dai parametri intrinseci (Fov, distanza focale e dimensione della finestra).

L'idea è analoga a scegliere l'obiettivo con cui inquadrare la scena.

Definisce la porzione di mondo inquadrata dal punto di vista già scelto.

Ci sono due proiezioni principali:

1. **Ortografica:** Il volume di vista è **parallelepipedo**. Gli oggetti non cambiano dimensione con la distanza.
2. **Prospettica:** il volume di vista è un **tronco di piramide**. Gli oggetti lontani appaiono più piccoli. Sarebbe la proiezione più realistica.

In entrambi i casi il **volume di vista** viene proiettato in un cubo di dimensioni $[-1,1]$ su tutti e tre gli assi. Questo cubo viene chiamato **volume di vista canonico**. La GPU lavora più facilmente usando sempre questo cubo standard.

La matrice della proiezione ortografica:

- prende il volume di vista
- lo scala
- lo trasla per trasformarlo nel cubo canonico

i parametri in input (right,left,top, bottom,near,far) definiscono i limiti del volume visibile.

$$\mathbf{M}_{orth} = \begin{bmatrix} \frac{2}{r-l} & 0 & 0 & -\frac{r+l}{r-l} \\ 0 & \frac{2}{t-b} & 0 & -\frac{t+b}{t-b} \\ 0 & 0 & \frac{2}{n-f} & -\frac{n+f}{n-f} \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Proiezione prospettica

Dobbiamo scalare entrambe le coordinate x e y

$$x_s = \frac{dx}{z} \quad y_s = \frac{dy}{z}$$

Queste formule ci dicono:

- più z aumenta (oggetto lontano)
- più la coordinata proiettata diventa piccola.

Quindi l'oggetto si rimpicciolisce.

Il problema è: x e y vengono scalati con la divisione per z , ma z cosa diventa?

Quello che vogliamo che $z = -n$ e $z = -f$ restino invariate.

$$z_s = n + f - \frac{nf}{z}$$

Il passo successivo è porre la distanza focale uguale alla distanza del piano vicino, (near plane) ovvero $d = -n$. Questo significa che l'immagine coincide esattamente con la base minore del volume di vista. Abbiamo però un problema, non possiamo fare una divisione. Quindi come possiamo rappresentare x/z con le matrici? usiamo le coordinate omogenee. Dopo la moltiplicazione per la matrice di proiezione, le coordinate vengono divise per w . Questo passaggio prende il nome di **divisione prospettica**. La matrice prospettica trasforma il tronco di piramide della camera nel volume canonico. Dopo la trasformazione e la divisione prospettica, tutti i punti visibili finiscono dentro il cubo canonico $[-1,1]$.

Proiezione prospettica risultato

L'idea sarebbe:

- **Distanza focale piccola:** grandangolo, esempio videogioco FPS
- **Distanza focale grande** sarebbe teleobiettivo, poca distorsione e profondità compressa
- **Distanza focale infinita** si arriva alla proiezione ortografica e nessuna prospettiva

🎯 Cos'è la divisione prospettica?

La divisione prospettica è il "trucco" matematico fondamentale che crea l'illusione della profondità 3D, ovvero fa in modo che gli oggetti più lontani ci appaiano più piccoli. PDF

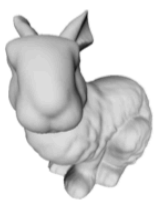
⚙️ Come funziona in modo intuitivo

Per gestire le trasformazioni geometriche, la pipeline grafica aggiunge alle classiche coordinate spaziali (X, Y, Z) una quarta coordinata "temporanea", detta coordinata omogenea. Nella proiezione prospettica, questa quarta coordinata viene usata per "memorizzare" quanto il punto è distante dall'osservatore. La divisione prospettica è semplicemente l'istante in cui il sistema divide tutte le coordinate del punto per questa distanza. PDF + 3

📦 Il risultato finale

Compiendo questa divisione, otteniamo due grandi risultati:

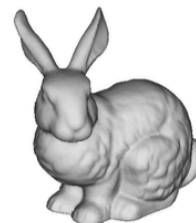
- **Visivo:** Si crea l'effetto di rimpicciolimento prospettico, deformando l'immagine in modo realistico. PDF
- **Geometrico:** Tutto lo spazio a forma di tronco di piramide che la telecamera sta inquadrando viene "compresso" in un cubo perfetto e standardizzato (il volume di vista canonico). Da questo cubo normalizzato, la scheda video avrà vita facilissima a proiettare l'immagine finale sul rettangolo 2D del nostro schermo. PDF + 4



d piccolo



d grande



d infinito
(proiezione ortogonale)



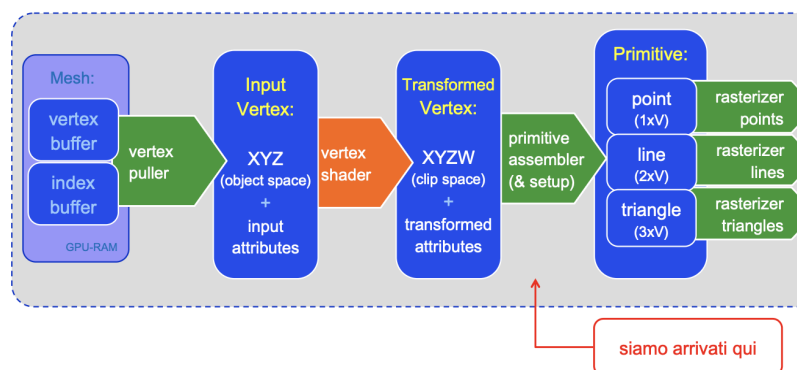
1. Viewport (Dove finisce l'immagine)

- **Cos'è:** La viewport è letteralmente la porzione della tua finestra di output (il rettangolo di pixel, es. 1920x1080) su cui andrai a stampare l'immagine finale. [PDF](#)
- **Cosa fa la trasformazione:** Prende i vertici che avevamo racchiuso nel famoso "cubo canonico" (il Sistema normalizzato con coordinate tra -1 e 1) e li converte nelle coordinate reali del tuo schermo. [PDF + 2](#)
- **La Matematica:** È un passaggio banalissimo. Si tratta solo di combinare una **traslazione** (per centrare l'immagine) e una **scalatura** (per adattarla ai pixel). A questo punto, l'immagine è praticamente in 2D e la profondità z viene solo "messa da parte" come attributo per ricordarsi quale oggetto copre l'altro. [PDF + 2](#)

2. Aspect Ratio (Non deformare la scena!)

- **Il problema:** L'aspect ratio è il rapporto tra la larghezza e l'altezza (w/h). L'effetto combinato di telecamera (proiezione) e schermo (viewport) rischia di **deformare orribilmente l'immagine** (es. allungandola o schiacciandola come uno specchio deformante). [PDF + 1](#)
- **La regola d'oro:** Per non avere distorsioni, il rapporto tra le dimensioni del tuo Volume di Vista (la finestra nel mondo 3D) deve essere **identico** al rapporto della tua Viewport (la finestra 2D fisica). Ovvero deve valere la proporzione $dy : dx = h : w$. [PDF + 1](#)
- **Cosa succede se l'utente ridimensiona la finestra?** Per mantenere l'immagine intatta e non deformata, il programma deve fare una di queste due cose: [PDF](#)
 - 1 **Modificare la viewport:** L'immagine viene rimpicciolita mantenendo le sue proporzioni originali (è il motivo per cui a volte compaiono le bande nere ai lati o sopra/sotto). [PDF](#)
 - 2 **Modificare il volume di vista:** Si cambia letteralmente l'inquadratura della telecamera, mostrando una porzione di mondo diversa (più larga o più stretta) senza schiacciare i modelli 3D. [PDF](#)

Rasterizzazione e operazioni per-frammento



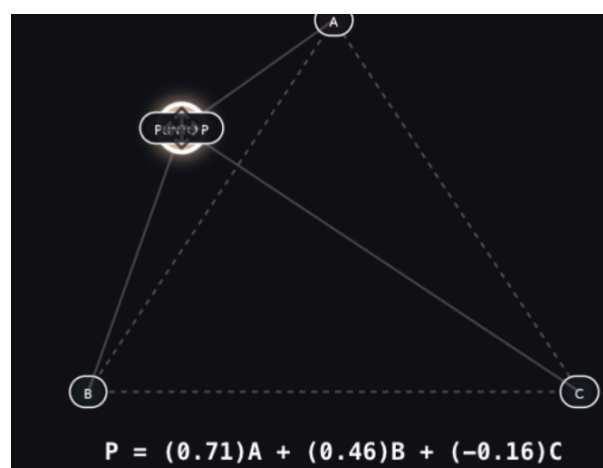
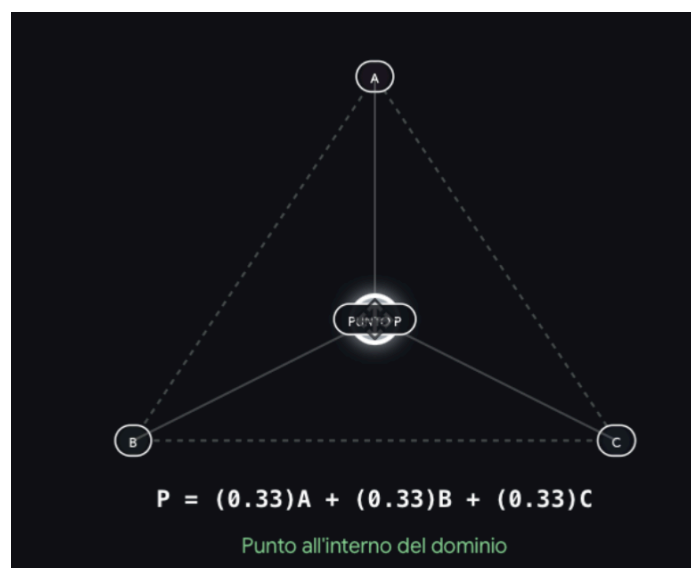
Assemblaggio primitive

Ricordiamo: i vertici sono in coordinate di **viewport**: **2D + profondità**.

A questo punto, i vertici vengono raggruppati per primitiva:

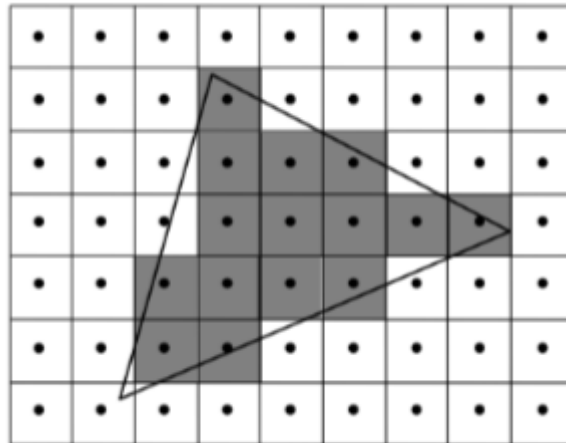
- **punto:** 1 vertice
- **segmento:** 2 vertici
- **triangolo:** 3 vertici

La rasterizzazione avviene su ogni primitiva in parallelo. La rasterizzazione le converte (le primitive) in **frammenti**. Per rasterizzare un triangolo, la GPU calcola la sua **bounding box** e utilizza le **coordinate baricentriche** per stabilire se un pixel si trova al suo interno. Queste coordinate sono fondamentali perché vengono usate anche come “pesi” per interpolare gli attributi noti dei vertici su ogni singolo frammento. In sostanza le coordinate baricentriche sono utili per capire se il punto che sto analizzando è dentro o fuori al triangolo, se anche solo uno di questi pesi diventa negativo allora significa che quel pixel preso in considerazione non va colorato.

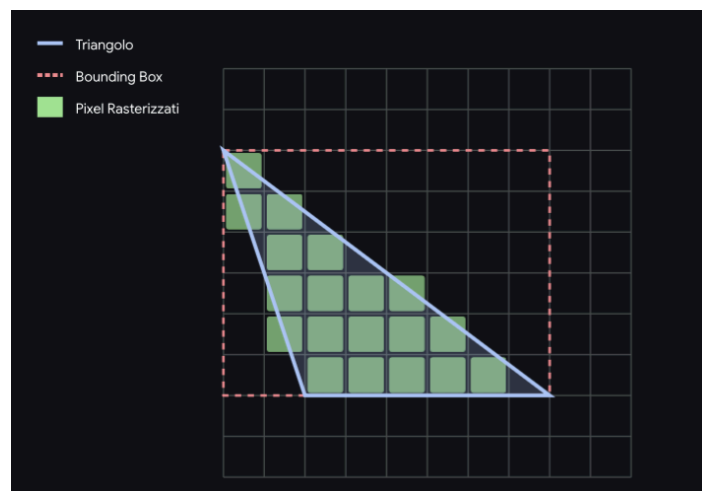
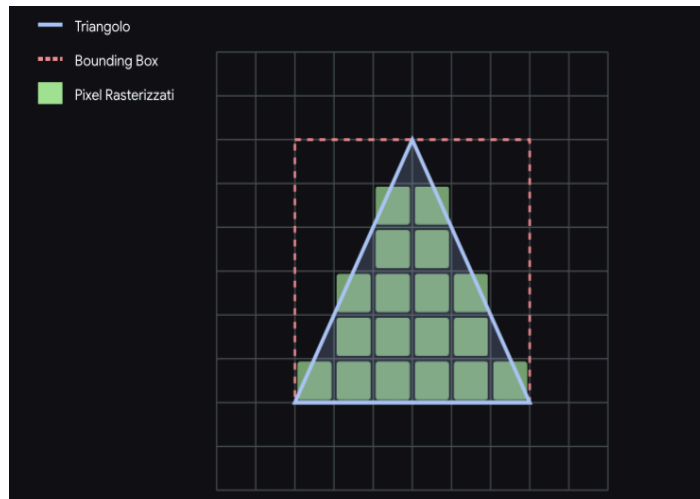


Fuori dal triangolo

Una **bounding box** è semplicemente il rettangolo più piccolo possibile che racchiude completamente una figura geometrica. **Nella rasterizzazione** è molto utile per non fare calcoli inutili, quando la GPU deve colorare un triangolo, invece di testare tutti i milioni di pixel restringe il campo e valuta solo i pixel contenuti nella sua bounding box.



NOTA: si ragiona SEMPRE per triangoli. Qualsiasi personaggio 3D è in realtà un “guscio” formato da migliaia o milioni di triangoli.

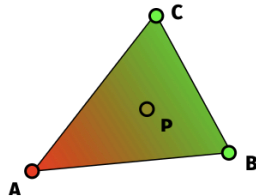


Ci sono due metodi principali per calcolare come la luce colpisce e colora un oggetto 3D:

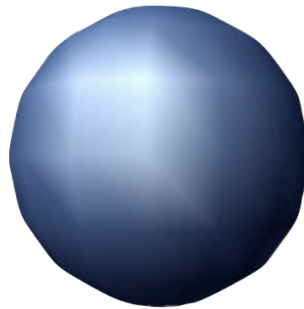
- **Illuminazione per-vertex:** il computer calcola la formula della luce solo nei 3 vertici del triangolo. Per colorare l'interno, si limita a fare una media (interpola) sfumando i tre colori dei vertici. **È un metodo veloce ma impreciso**
- **Illuminazione per-pixel:** molto più precisa e realistica. Invece di calcolare il colore subito, il computer interpola le coordinate e le direzioni su tutta la superficie, e poi applica la formula della luce per ogni singolo pixel che compone il frammento. **È più costoso in termini di calcolo.**

In maniera semplice: In pratica, il sistema calcola l'inclinazione e la posizione esatta nello spazio per ogni singolo pixel (o frammento). Ora che il sistema conosce l'inclinazione (normale) e la posizione esatta di quel pixel specifico, prende l'equazione dell'illuminazione (ad esempio quella con componente diffusa e speculare) e la risolve interamente solo per quel pixel per trovare il suo colore finale. Questo processo viene ripetuto per tutti i pixel che compongono l'oggetto.

Interpolazione del colore

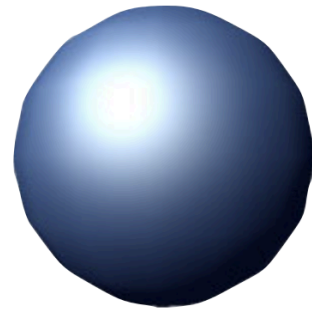


Illuminazione
per-vertice



Valuta colore sui vertici,
interpola sui pixel

Illuminazione
per-pixel



Interpola posizioni e normali,
calcola il colore per pixel

<http://www.cgchannel.com/2010/11/cg-science-for-artists-part-2-the-real-time-rendering-pipeline/>

11/1/68 - 2025/26 - Fondamenti di Computer Grafica - Enrico D'Amico

In parole povere, l'**interpolazione** è un'operazione matematica che serve a calcolare (o stimare) un valore "sconosciuto" che si trova in mezzo a dei valori "noti".

Nel contesto della grafica 3D e del tuo esame, ecco come funziona: [🔗](#)

📐 **I Valori Noti:** La GPU conosce i dati (come colore o inclinazione/normale) solo nei tre **vertici** agli angoli del triangolo.

🎨 **I Valori Sconosciuti:** Il triangolo rasterizzato è riempito da tanti pixel interni (frammenti) di cui la scheda video inizialmente non sa nulla.

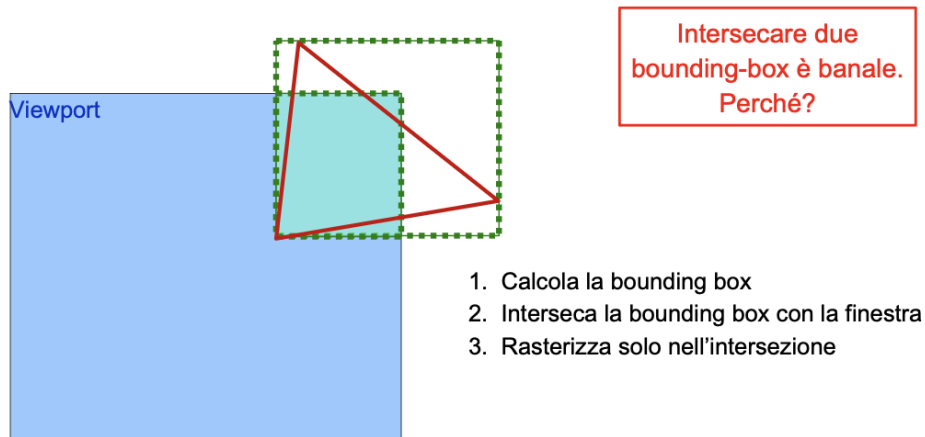
⚖️ **Il Calcolo (Coordinate Baricentriche):** Per dare un valore a un pixel interno, il sistema usa una media pesata basata sulle **coordinate baricentriche** (chiamate α , β , γ). Più un pixel è vicino a un vertice, maggiore sarà la percentuale (il "peso") che prenderà dal valore di quel vertice. [🔗 +4](#)

💡 **Due esempi pratici visti finora:**

- **Colore (Illuminazione per-vertice):** Se il vertice A è calcolato come rosso e il vertice B come giallo, l'interpolazione fa in modo che i pixel in mezzo sfumino gradualmente diventando arancioni. [🔗 +1](#)
- **Luce (Illuminazione per-pixel):** Invece del colore finale, si interpolano le *posizioni* e le *normali* (le inclinazioni) dai vertici ai pixel interni, per poi calcolare la complessa equazione della luce su ogni singolo pixel interpolato. [🔗](#)

DA RICORDARE: l'interpolazione è il calcolo matematico che sfrutta le coordinate baricentriche per andare ad assegnare il colore

Clipping



Operazioni per frammento

I frammenti sono generati dalla **rasterizzazione**. Ogni frammento è caratterizzato da:

- **una posizione (i,j)** in coordinate intere nel sistema della finestra
- **un insieme di attributi** sono informazioni essenziali per il **rendering** (colore, profondità ecc)

Fragment shader

È un programma eseguito in parallelo direttamente sulla **GPU**. Il suo compito principale è elaborare ciascun **frammento** in modo totalmente indipendente dagli altri.

Input:

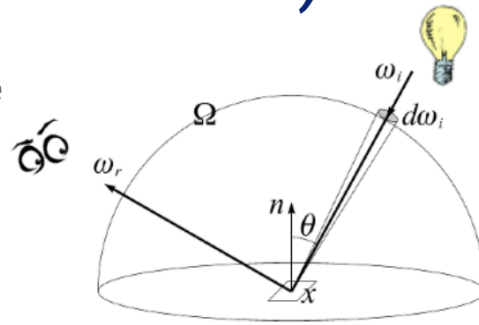
- **coordinate (i,j)** del frammento
- **attributi del frammento**(coordinate baricentriche)

output: attributi che sono destinati al pixel di coordinate (i,j) del framebuffer.

Il dato più importante che esce da questa fase è il colore finale che dovrà essere assegnato a quel pixel sullo schermo.

Shading (modello fisico)

- Modellare accuratamente l'interazione tra luce e superficie degli oggetti è complesso e costoso da calcolare
- Equazione della radianza:



$$L_r(x, \vec{\omega}_r) = \int_{\vec{\omega}_i \in \Omega} f_r(x, \vec{\omega}_i, \vec{\omega}_r) L_i(x, \vec{\omega}_i) (\vec{\omega}_i \cdot \vec{n}) d\vec{\omega}_i$$

luce emanata da x
in direzione ω_r

Funzione di Distribuzione
Bidirezionale della Riflettanza
(BRDF)
(proprietà del materiale)

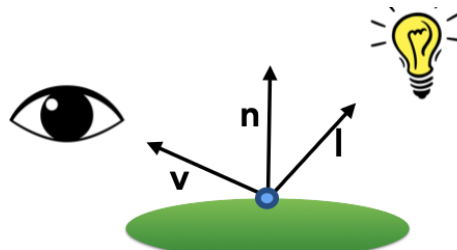
luce ricevuta
da x dalla
direzione ω_i

modulazione
geometrica
(Lambert)

Il modello fisico reale della luce (equazione della radianza) è troppo complesso e costoso da calcolare in tempo reale. Per questo motivo usiamo **un'approssimazione forfettaria** divisa in **tre parti** che poi vengono sommate per ottenere il colore finale.

Prima di calcolare la luce, il computer ha bisogno di tre **direzioni(vettori)**:

- **l (luce)**: la direzione da cui proviene l'illuminazione
- **v(vista)**: la direzione che punta verso l'osservatore/telecamera
- **n(normale)**: il vettore perpendicolare alla superficie dell'oggetto



Le tre componenti dello shading:

1. **Shading d'ambiente**: è la luce base della scena, per evitare che le parti in ombra siano completamente nere. Anche se ci sono 10 luci diverse nella scena, il contributo ambientale si somma una volta sola.
2. **Shading diffuso**: simula la luce sulle superfici **opache**. Dipende tutto dall'angolo di incidenza: **più la luce (l) colpisce la superficie in modo perpendicolare (n) più è illuminata**. Cosa importante che l'osservatore non c'entra nulla (non influisce in nessun modo)
3. **Shading speculare**: **simula i materiali lucidi**. Questa componente dipende fortemente dalla posizione dell'osservatore(v). L'idea è che usa l'esponente di **Phong(p)** che più è alto più il materiale è lucido.

Unendo questi 3 pezzi si ottiene la formula dello shading:

$$L = k_a I_a + k_d I \max(0, \mathbf{n} \cdot \mathbf{l}) + k_s I \max(0, \mathbf{n} \cdot \mathbf{h})^p$$

Ambiente Diffusa Speculare

Se ci sono diverse luci, si sommano i loro contributi.
La luce ambiente va considerata una volta sola.

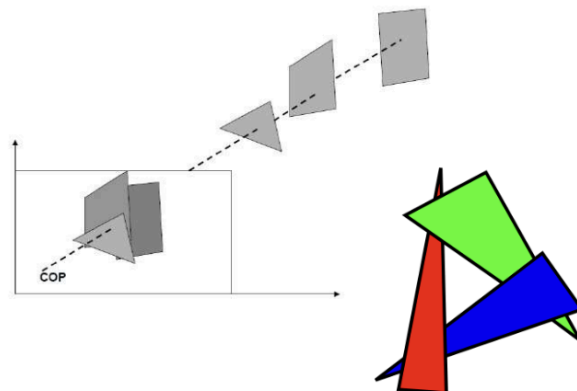
Output combiner

Gli attributi generati dal fragment shader passano in una fase finale, che genera i valori da assegnare ai vari strati del framebuffer:

- Color buffer: memorizza il colore finale di ogni pixel
- Depth buffer
- Stencil buffer

Ordinamento in profondità

- Gli oggetti, a meno che siano semi-trasparenti, occludono quelli dietro
- Ordinare gli oggetti per profondità e fare il rendering dal più lontano al più vicino
- Non è sempre possibile!



questi non si
possono ordinare

Ordinamento in profondità in poche parole :

1. Cos'è? (L'Algoritmo del Pittore)

L'ordinamento in profondità (chiamato storicamente *Painter's Algorithm*) è il metodo più antico e intuitivo per gestire la visibilità. Funziona letteralmente come un pittore davanti a una tela: disegna prima le montagne in lontananza, poi la casa davanti, e infine l'albero in primissimo piano. In questo modo, ogni volta che disegni qualcosa di nuovo, "sovrascrivi" e copri ciò che sta dietro.

2. A cosa serve?

Serve a risolvere le occlusioni: nascondere ciò che non deve essere visto. Inoltre (e ricordalo per l'esame!), **è assolutamente necessario per l'Alpha Blending (le trasparenze)**. Per mescolare correttamente le vernici semitrasparenti usando la formula vista nelle slide, *devi per forza* dipingerle in ordine di distanza dal più lontano al più vicino.

3. Perché "non è sempre possibile"? (Il limite fatale)

Questo metodo ha un difetto enorme: ragiona per **intere primitive** (es. valuta un intero triangolo). Ma cosa succede in questi due casi molto comuni nel 3D?

- **Compenetrazione:** Immagina due triangoli di carta che si incastrano a forma di "X". Metà del triangolo A sta davanti a B, e metà del triangolo B sta davanti ad A. Chi disegni per primo? Impossibile dirlo!
- **Sovrapposizione Ciclica:** Il triangolo A copre B, B copre C... e C copre A! (Come nel sasso-carta-forbice). Nessuno dei tre è "il più lontano" in assoluto. L'algoritmo va in tilt.

La soluzione sarebbe lo **z-buffering**.

✓ La soluzione: Z-buffering

- Invece di provare a ordinare interi oggetti (operazione difficile), lo Z-buffer lavora a livello di **singolo pixel**. [↗](#)
- Per ogni pixel, la GPU memorizza la profondità (Z) del frammento più vicino trovato finora. [↗](#)
- **Perché "funziona sempre"?** Perché non gli importa dell'ordine degli oggetti: ogni volta che arriva un nuovo "pezzettino" (frammento), il computer guarda solo se quel pezzettino è più vicino di quello che c'è già. [↗](#)
- È la soluzione standard nelle schede video perché è facile, veloce e risolve i casi impossibili dell'ordinamento. [↗](#)

⚠ L'unico limite

- L'unica cosa che lo Z-buffer non può risolvere da solo sono le **semitrasparenze** (alpha-blending), dove l'ordine di disegno torna a essere fondamentale per mescolare bene i colori. [↗ +2](#)

🎨 Cos'è l'Alpha-Blending?

È la tecnica utilizzata per gestire la **semitrasparenza** degli oggetti. Mentre i colori standard hanno tre componenti (RGB), qui entra in gioco il **canale alpha (α)**, che indica quanto un frammento è opaco. [↗ +1](#)

- $\alpha = 0$: totalmente trasparente (invisibile). [↗](#)
- $\alpha = 1$: totalmente opaco (copre tutto ciò che sta dietro). [↗](#)

⚠ Il "Punto Debole" (Fondamentale per l'esame)

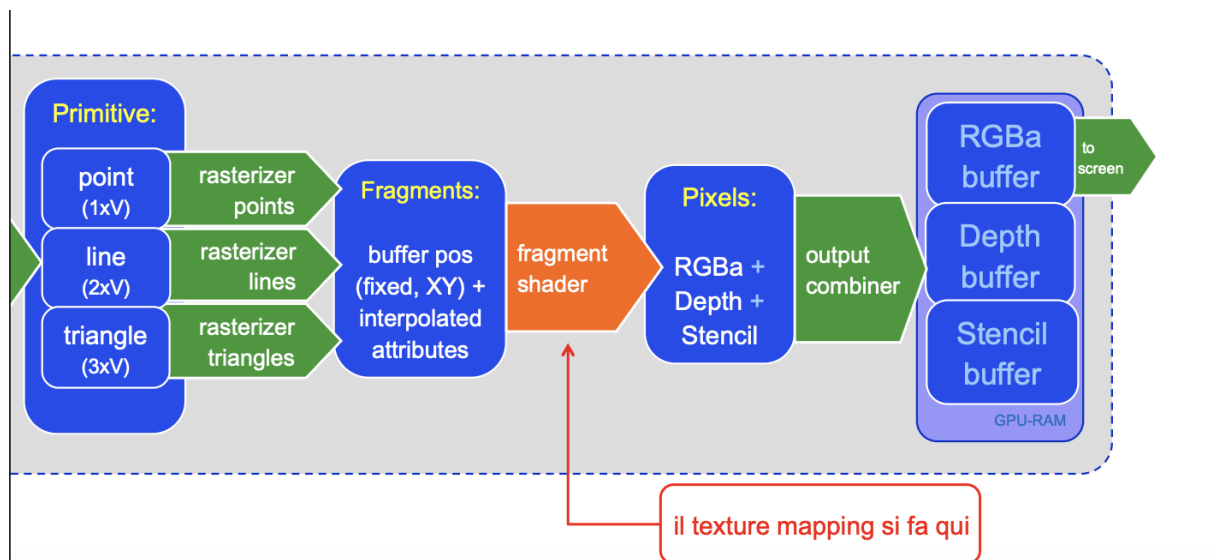
A differenza dello Z-buffer, l'Alpha-Blending ha un grosso limite:

1. **Richiede l'ordinamento:** i frammenti devono essere visualizzati rigorosamente **in ordine di distanza** (dal più lontano al più vicino) per mescolarsi correttamente. 🔗
2. **Conflitto con lo Z-buffer:** il solo depth buffer non riesce a gestire correttamente la situazione se ci sono più di due frammenti semitrasparenti che si sovrappongono. 🔗

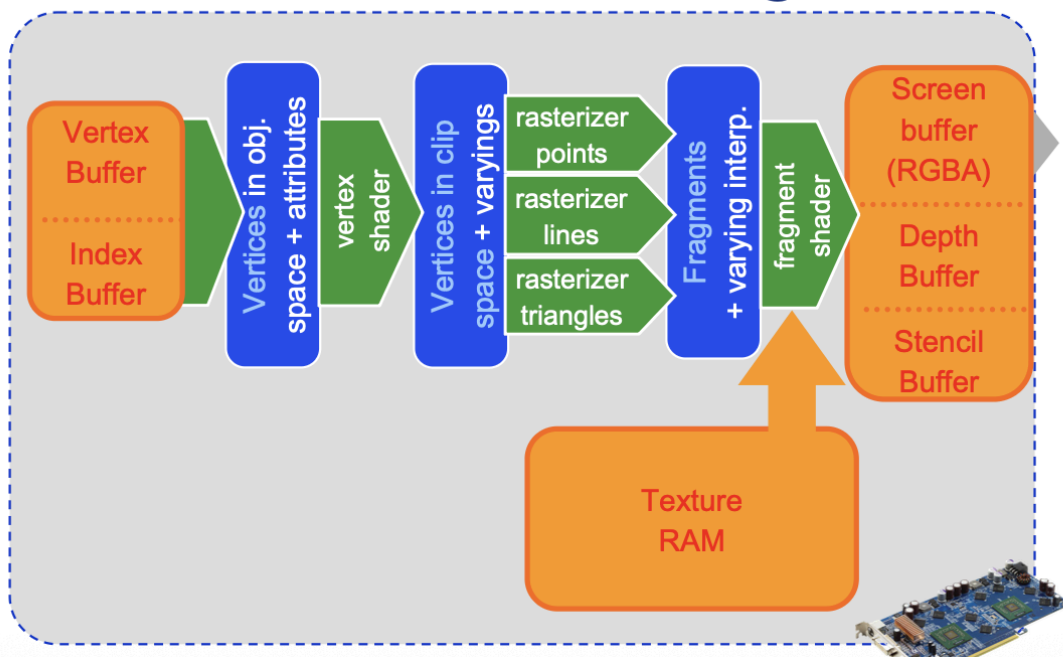
🎯 **In sintesi:** Lo Z-buffer risolve le occlusioni tra oggetti opachi senza bisogno di ordinarli, mentre l'Alpha-Blending serve per la trasparenza ma ti obbliga a ordinare gli oggetti dal più lontano al più vicino.

Texture mapping

Analogamente al caso 2D, si riferisce all'applicazione di texture alle primitive. Col texture mapping si possono fare molte operazioni sofisticate ma noi ci soffermeremo solo nel caso dei triangoli.



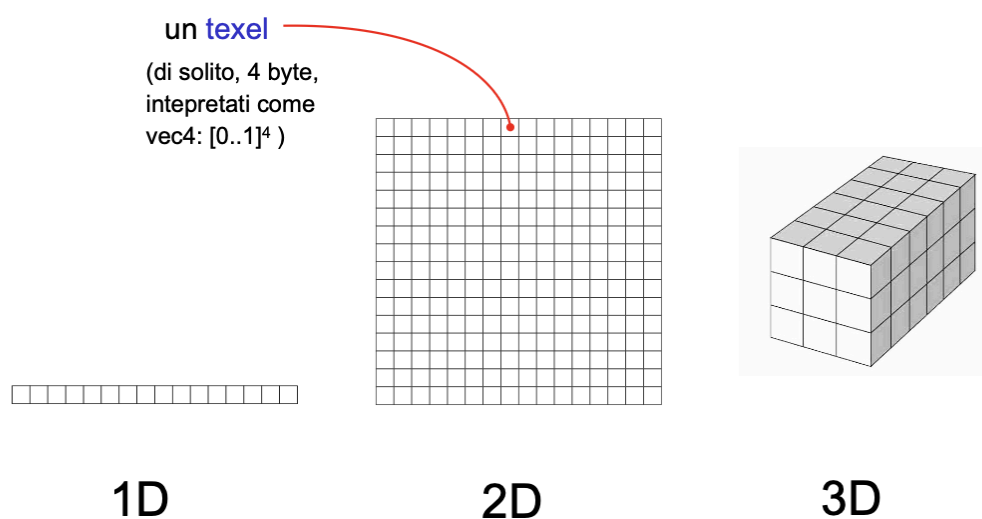
RAM della scheda grafica



La **Texture RAM** fa parte della **video RAM**. Si tratta di una memoria condivisa tra tutti i frammenti ed è gestita in modi specifici. Accessibile agli shader con regole di accesso speciali, **cosa molto importante che è scrivibile dal programma CPU ma read only per gli shader.**

La memoria è organizzata in “texture sheet” ovvero griglie regolari di elementi elementari chiamati **texel** (texture element).

Queste griglie possono essere strutturate in 1D, 2D (caso più comune) oppure in 3D.



Un texel di solito occupa 4 byte e viene interpretato come un vec4.

Un **texel** è un vettore composto da un numero **N di float** dove $N = 1, 2, 3$ o 4 canali della texture. Il formato a **4 canali** è il caso più comune. (negli shader, ogni accesso restituisce un **vec4**). I dati possono essere memorizzati fisicamente in vari formati a 8 bit fixed point (caso più comune), a 16 bit fixed point o 32 bit float.

Una **texture** ha una data risoluzione per esempio 1024×1024 (caso 2D) oppure $32 \times 64 \times 32$ (caso 3D). Il **numero di texel per texture è limitato**, la risoluzione massima per dimensione dipende dall'implementazione ma di norma è tra 2048 e 4096.

Le texture caricate in memoria video prima del rendering (attraverso il bus) è costoso.

Texture: accesso

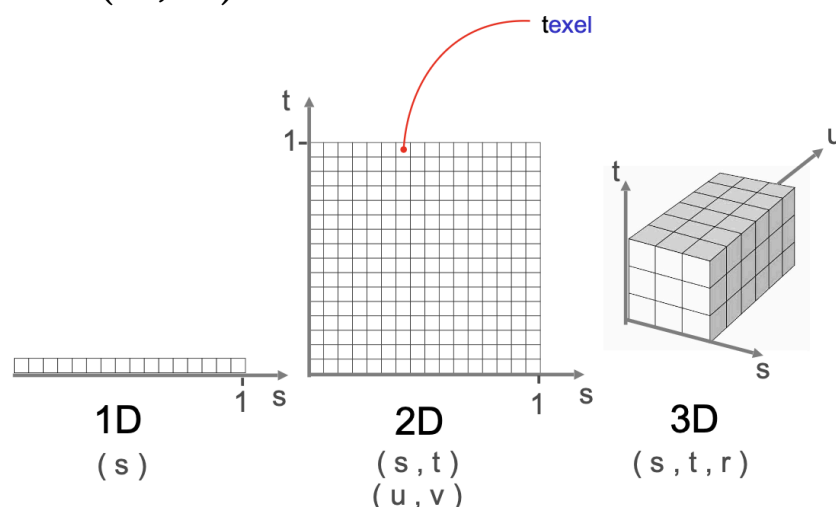
L'accesso a una texture è analogo al **campionamento** di un'immagine.

Il **texture fetch** (look-up) è l'operazione con cui la scheda video "legge" il colore della texture per applicarlo ai pixel. La **RAM della CPU** usa numeri interi per trovare i dati, mentre **l'accesso alle texture** usa **coordinate continue** con numeri decimali (float). L'accesso produce valori attraverso **interpolazione lineare**.

Sistema di coordinate

Le texture usano un sistema di riferimento dedicato per orientarsi sull'immagine. Il nome delle variabili: si usano le lettere s, t, r oppure (u, v) per indicare gli assi. Una texture 2D usa le coordinate (s, t) .

Le coordinate vanno sempre da 0.0 a 1.0 in modo totalmente indipendente alla grandezza dell'immagine. Che la texture sia 512×512 o 1024×1024 il **centro sarà esattamente a $(0.5, 0.5)$**



Texture uso tipico

Perché sono state inventate le texture? per rispondere a questa domanda: *come facciamo ad aggiungere dettagli complessi (colori, loghi, rugosità) a un oggetto 3D?*

Abbiamo due strade (per memorizzare le texture):

1. **Salvare i dati nei vertici:** l'idea sarebbe assegnare il colore direttamente ai vertici del triangolo (spigoli). Il problema è che saremmo obbligati a creare migliaia e migliaia di triangoli minuscoli. **Per segnali complessi potremmo aver bisogno di moltissimi vertici ma abbiamo pochi campioni.**
2. **Usare una texture 2D:** Invece di creare una nuova geometria, si prende un'immagine piatta (texture) e la si "incolla" sopra i triangoli. Per la scheda video è un'operazione economica. La texture è una griglia regolare e ordinate. Abbiamo molti campioni (1 per texel).

Texture Mapping

Costruzione dell'UV-mapping

- Tipicamente svolto durante la modellazione
 - programmi semi-automatici
 - tool professionali di modellazione (richiedono competenza artistica)
- In pratica: dobbiamo piazzare ogni triangolo della mesh nello spazio della texture
- Analogo a:
 - sbucciare un frutto
 - sistemare i pezzi di buccia dentro un rettangolo
- Di solito è necessario tagliare la superficie:
 - genera discontinuità dei valori u, v (vertex seam)

Texture Mapping (La Teoria)

Questa slide spiega *come* colleghiamo matematicamente il 3D al 2D.

- **Parametrizzazione:** Dobbiamo creare una mappa che dica "questo punto del modello 3D corrisponde a quest'altro punto sull'immagine 2D".
- **U-V Mapping:** Il trucco consiste nell'assegnare a **ogni singolo vertice** del modello 3D una coppia di coordinate 2D, chiamate (s, t) oppure (u, v) . È come mettere un "pin" sul vertice che dice alla scheda video l'esatta coordinata $[0..1]$ da cui pescare il colore sull'immagine.

se ho capito bene l'idea del texture mapping sarebbe quella di andare ad assegnare dei punti 2D nella mia superficie 3D, lo faccio tramite UV Mapping che prende ogni triangolo della mesh nello spazio texture, quindi ogni singolo pezzo della texture la vado incollare nel mi...

Mostra il ragionamento

Esattamente! Hai capito la logica alla perfezione.

Hai descritto in modo molto intuitivo e corretto il processo dell'**UV Mapping**. Per essere impeccabile all'esame, ecco solo una piccola rifinitura tecnica di quello che hai appena detto:

1. L'assegnazione ai Vertici

Non assegni le coordinate 2D a tutta la superficie a caso, ma assegni una specifica coppia di coordinate 2D (s, t) **direttamente e solo ai 3 vertici** di ogni triangolo 3D della tua mesh.

2. Lo Spazio Texture (Il ritaglio)

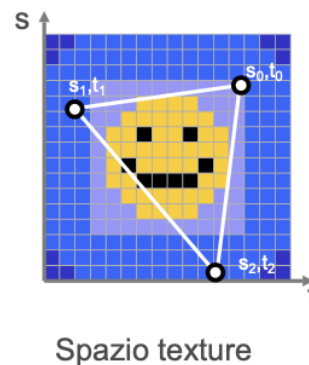
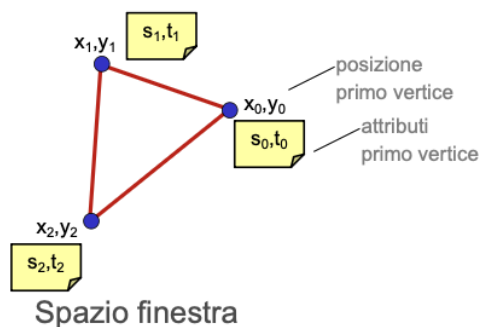
Così facendo, è come se dicessi al computer: "Il vertice A del triangolo 3D corrisponde al punto A sull'immagine 2D, il vertice B al punto B, e il vertice C al punto C". Di fatto, stai ritagliando un triangolino di "carta da parati" dall'immagine originale.

3. L'incollaggio finale (L'interpolazione)

Quando la scheda video deve colorare il triangolo sullo schermo, usa le **coordinate baricentriche** (che hai studiato prima!) per calcolare in automatico e sfumare le coordinate (s, t) su tutti i pixel che stanno *all'interno* del triangolo. Così facendo, incolla perfettamente l'immagine pezzettino per pezzettino sul tuo modello 3D.

Texture Mapping

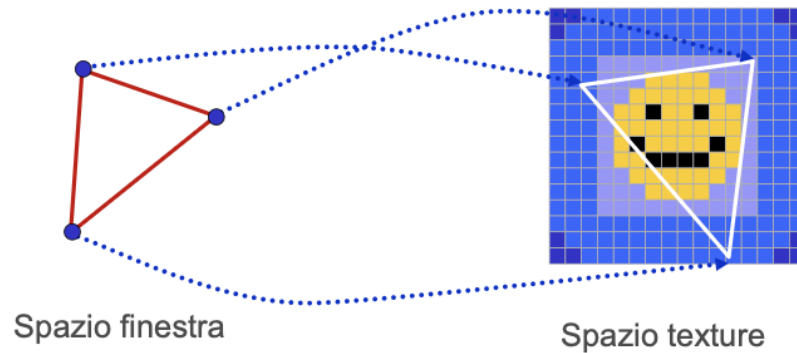
- A ogni **vertice** (di ogni triangolo) assegniamo le coordinate s, t nello **spazio texture**



Stiamo ad ogni coordinata (x, y) quale pezzo di texture deve andare a leggere.

Texture Mapping

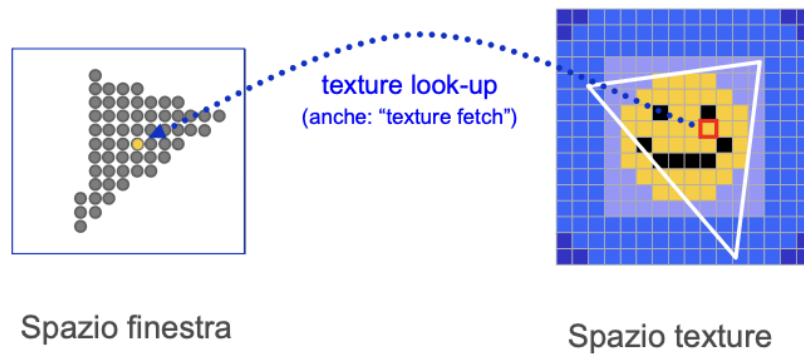
- Stiamo definendo una **mappa** tra il triangolo 3D e un triangolo sulla texture



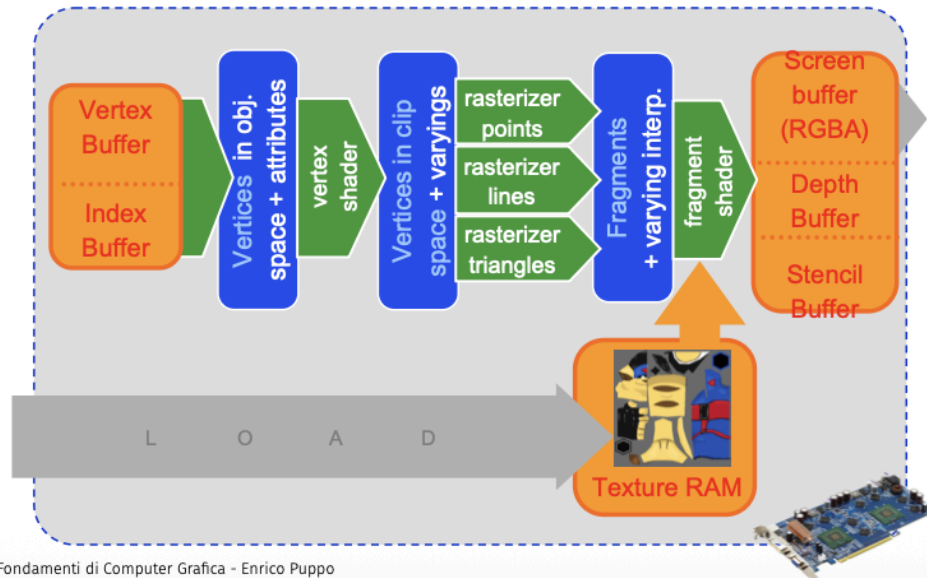
114468 - 2025/26 - Fondamenti di Computer Grafica - Enrico Puppo

Texture Mapping

- Ogni **frammento** ha le proprie coordinate s, t in **spazio texture**

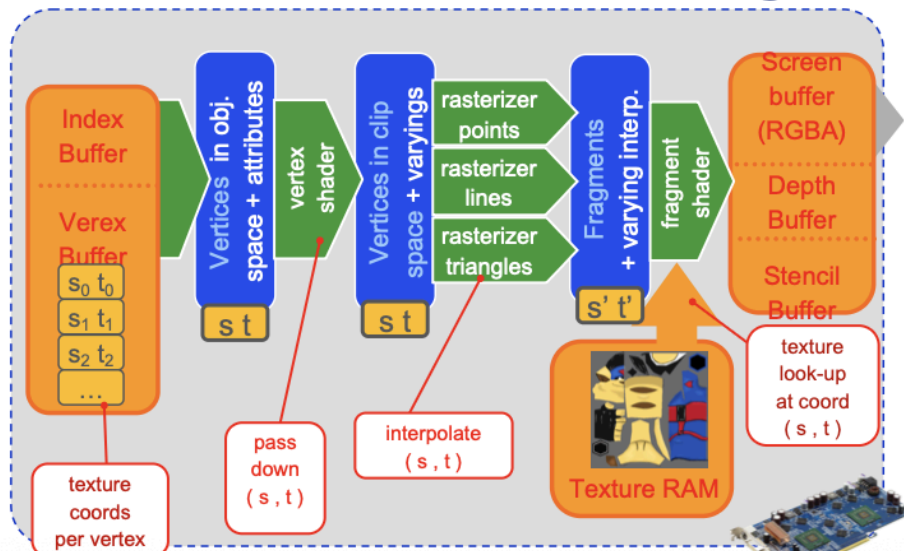


Prima del rendering



114468 - 2025/26 - Fondamenti di Computer Grafica - Enrico Puppo

Durante il rendering



1. Prima del rendering (Fase preparatoria)

- Il professore ti fa vedere una freccia grigia gigante con scritto "LOAD". 
- Vuole dirti che caricare le immagini dal PC alla scheda video è un'operazione pesante. Per questo, le texture vengono pre-caricate in blocco nella Texture RAM prima che la GPU inizi a disegnare il primissimo frame.  +2

2. Durante il rendering (Il viaggio delle coordinate)

Qui ti fa vedere il "viaggio" delle coordinate (s, t) attraverso i vari stadi della GPU: 

-  **Vertex Buffer:** Le coordinate nascono qui. Vengono lette dalla memoria iniziale perché sono salvate e associate a ogni singolo vertice $(s_0, t_0, s_1, t_1, \text{ecc.})$.  +1
-  **Vertex Shader:** Qui non subiscono modifiche. Il box rosso "pass down (s, t) " significa che il Vertex Shader si limita a prendere queste coordinate e a "passarle giù" al livello successivo intatte. 
-  **Rasterizer:** È qui che avviene l'interpolazione! Il box "interpolate (s, t) " fa esattamente quello che dicevamo prima: usa le coordinate baricentriche per calcolare i valori intermedi (che lui chiama s', t') per ogni singolo frammento.  +2
-  **Fragment Shader:** Arrivati alla fine, ogni frammento usa le sue coordinate s', t' appena calcolate per fare il famoso "texture look-up" (o fetch). Legge il colore dalla Texture RAM e decide il colore finale del pixel.  +3

2. Tipi di Mappe (Non solo colori!)

Un errore comune è pensare che le texture siano solo le "foto" colorate degli oggetti. In realtà, una texture è un contenitore di dati  5. Le più usate sono:

- **Color Map (o Diffuse):** La classica immagine colorata RGB.
- **Alpha Map:** Un'immagine in bianco e nero che dice al computer quali parti dell'oggetto sono trasparenti (0) e quali opache (1). *Esempio:* Si usa per fare i fili d'erba, le foglie degli alberi o le pellicce senza modellare ogni singolo pelo geometricamente  6  7.
- **Bump / Normal Map:** Un'immagine speciale che "inganna" il calcolo della luce (lo shading di cui parlavamo prima) facendo credere che una superficie liscia abbia in realtà delle rugosità o dei rilievi  5  7.

Cosa succede se a un vertice diamo una coordinata texture maggiore di 1 (cioè fuori dall'immagine)? La GPU offre due soluzioni ⁹ :

- **Clamp:** Il computer prende il colore dell'ultimo pixel sul bordo e lo "stira" all'infinito.
- **Repeat:** L'immagine ricomincia da capo (fa un *loop*). Questo è utilissimo per fare un muro di mattoni lunghissimo usando una texture piccolissima! Bisogna solo assicurarsi che l'immagine sia "Tileable" (cioè che il bordo destro combaci perfettamente con quello sinistro, senza far notare lo stacco) ⁹ ¹⁰ .

5. Magnificazione vs Minificazione

Questo è il cuore logico della lezione. Quando ci muoviamo nel gioco, la telecamera si avvicina o si allontana dal muro di mattoni. Il computer deve capire in che situazione si trova per calcolare il colore corretto ¹⁰ :

- **Magnificazione:** Siamo vicinissimi al muro. Pochi Texel dell'immagine devono riempire tantissimi Pixel dello schermo. Stiamo "ingrandendo" l'immagine ¹¹ .
- **Minificazione:** Siamo lontanissimi. Tantissimi Texel (es. un'intera texture 1000x1000) devono essere schiacciati in un solo piccolo Pixel dello schermo. Stiamo "rimpicciolendo" ¹¹ .

La chicca per l'orale: Come fa il Fragment Shader a sapere se sta ingrandendo o rimpicciolendo? Usa le **Derivate in spazio finestra**. Poiché la scheda video processa i frammenti a gruppi (in parallelo), il programma fa un rapido confronto "sbirciando" le coordinate del frammento vicino. Se la differenza è minima, sto ingrandendo. Se la differenza è enorme, sto rimpicciolendo ¹² ¹³ .

6. I Filtri (Cosa fare in Magnificazione)

Se siamo in magnificazione (molto vicini), dobbiamo "stirare" la texture. Ci sono due filtri principali ¹⁴ :

- **Nearest Filtering (Il più economico):** Il computer prende brutalmente il colore del Texel più vicino. *Risultato:* L'immagine diventa a quadrettoni (effetto "Minecraft" o "Texellazione").
- **Bilinear Interpolation:** Il computer prende i 4 Texel più vicini e ne fa una media sfumata. *Risultato:* L'immagine è fluida, ma può sembrare un po' sfuocata se ci si avvicina troppo ¹⁴ .

7. Il trucco per la Minificazione: MIP-Mapping

Se siamo lontanissimi (Minificazione), l'interpolazione bilineare fallisce. Immagina una scacchiera bianca e nera lontanissima: se il computer "salta" dei texel per pescarne 4 a caso, potrebbe beccare solo pixel neri, o solo bianchi, creando uno sfarfallio bruttissimo (chiamato aliasing) ¹⁶.

La soluzione geniale: Il MIP-Mapping ("Molto in poco spazio"). Invece di far calcolare al computer la media di 1000 pixel rimpiccioliti in tempo reale, la GPU si prepara prima. Genera una serie di copie della stessa texture, ognuna grande la metà della precedente (Livello 0 = originale, Livello 1 = metà, Livello 2 = un quarto, ecc.) ¹⁶. Durante il rendering, in base alla distanza (e calcolando la derivata per capire la proporzione Pixel/Texel), **il computer sceglie matematicamente quale versione rimpicciolita della texture usare** ¹⁶. Risparmia calcoli enormi ed elimina lo sfarfallio visivo ¹⁷.

La magnificazione avviene quando un texel occupa tanti pixel, quindi l'immagine può risultare pixelata. Quindi ci sono due modi per risolverlo:

il primo fa che l'immagine risulti pixelata. L'altro spalma il colore dei texel rendendo la texture più uniforme ma con una risoluzione più bassa.

Minificazione: processo inverso, ovvero tanti texel occupano un singolo pixel. E unico modo per risolvere è il mip mapping, ovvero lo fa direttamente la GPU -> prende la texture originale e dimezza la grandezza, e lo ripete finché non arriva ad avere un solo texel. Il computer quando andiamo a dezoommare decide quali di queste texture applicare.

LAB3

Il nostro obiettivo principale è quello di crearci le struct principali. Prima andiamo a dichiarare due costanti:

```
// ball
const float ball_radius = 10.0;
// paddle
const sf::Vector2f paddle_size = {100.0, 16.0};
```

Queste ci servono per fissare le dimensioni di pallina e racchetta. Ora creiamo le 3 struct:

```
struct Ball
{
    float radius;
    sf::Vector2f pos;

    Ball ();
```

```

    void draw (sf::RenderWindow& window);
};

struct Paddle
{
    sf::Vector2f size;
    sf::Vector2f pos;

    Paddle ();
    void draw (sf::RenderWindow& window);
};

struct State
{
    Ball ball;
    Paddle paddle;

    State () {}
    void draw (sf::RenderWindow& window);
};

```

Ora implementiamo i costruttori:

todo

02a_move_ball

Come da testo aggiungiamo due variabili globali che serviranno per la velocità e direzione (di rilascio) della pallina:

```

float ball_speed = 300;
sf::Angle ball_start_angle = sf::degrees(-70);

```

alla **struct Ball** dobbiamo aggiungere due membri: un float per la velocità e un **sf::Angle** per la direzione di movimento della palla, che inizializziamo rispettivamente a **ball_speed** e **ball_start_angle** nel costruttore.

```

struct Ball
{
    float radius;
    float speed;

    sf::Vector2f pos;
    sf::Texture texture;
    sf::Angle angle;
}

```

```

    Ball();
    void draw(sf::RenderWindow& window);
    void move (float elapsed);
};

```

Abbiamo aggiunto un metodo move con un float che indica un lasso di tempo. Questo metodo ha il compito di muovere la pallina andando ad aggiornare la sua posizione precedente. Il metodo si implementa così:

```

void Ball::move(float elapsed)
{
    sf::Vector2f displacement(speed * elapsed, angle);
    pos += displacement;
}

```

La funzione move calcola lo spostamento moltiplicando la velocità per il tempo trascorso (speed * elapsed). È la formula fisica base: *Spazio = Velocità × Tempo*. Questo garantisce un movimento fluido.

Andiamo ad aggiungere a State un membro booleano start che indica se la palla è in movimento oppure no:

```

struct State
{
    Ball ball;
    Paddle paddle;
    bool start;

    State () : ball(),
               paddle(),
               start(false) {}
    void draw (sf::RenderWindow& window);
    void update(float elapsed);
};

void State::update(float elapsed)
{
    if(!start)
    {
        return;
    }
    ball.move(elapsed);
}

```

Il metodo **update** controlla se la pallina è in movimento oppure no. Ora dobbiamo mettere alle handle una callback che gestisce l'evento **sf::Event::KeyPressed** e

prende come parametro anche lo stato. Se premiamo lo space allora setta **start** di State a true. Aggiungiamo inoltre la callback generica **template void handle (T& event, State& state) {}** per gestire tutti gli eventi non riconosciuti.

```
template <typename T>
void handle(T& event, State& state){}

void handle(const sf::Event::KeyPressed& key, State& state)
{
    switch(key.scancode) {
        case sf::Keyboard::Space:
            state.start = true;
            return;
        default:
            return;
    }
}
```

Lo switch controlla quale tasto è stato premuto. Se viene premuto space allora viene attivata la pallina mettendo state.start a true, in caso negativo non fa nulla. Dobbiamo aggiornare il main:

```
int main()
{
    sf::RenderWindow window (sf::VideoMode ({window_width, window_height}),
window_title);
    window.setFramerateLimit (max_frame_rate);
    window.setMinimumSize(window.getSize());

    State state;
    sf::Clock clock;

    while (window.isOpen())
    {
        // events
        window.handleEvents (
            [&window] (const sf::Event::Closed&) { handle_close
(window); },
            [&window] (const sf::Event::Resized& event) {
handle_resize (event, window); },
            [&state] (const auto& event) { handle (event, state); }
        );

        // update
        state.update (clock.restart().asSeconds());
    }
}
```

```

        // display
        window.clear (sf::Color::Black);
        state.draw (window);
        window.display ();
    }
}

```

Il main rappresenta il classico Game Loop. Abbiamo aggiunto **sf::Clock::clock** che è appunto un cronometro, è importante per calcolare il delta time ovvero il tempo trascorso tra un frame e l'altro. Moltiplicando la velocità per il tempo del clock, la palla viaggerà sempre alla stessa velocità visiva su qualsiasi computer. Adesso dobbiamo gestire i limiti dello schermo. Andiamo ad aggiornare lo struct di State aggiungendo due funzioni nuove:

```

        void field_limits ();
        void collisions ();

```

Ora creiamo due funzioni per calcolare il nuovo angolo di rimbalzo della palla quando colpisce un ostacolo:

```

sf::Angle reflect_horizontal(sf::Angle a)
{
    sf::Vector2f v(1.0, a);
    v.x = -v.x;
    return v.angle;
}

sf::Angle reflect_vertical()
{
    sf::Vector2f v (1.0, a);
    v.y = -v.y;
    return v.angle();
}

```

reflect horizontal serve quando colpisce un muro (a sinistra o destra). Prende l'angolo attuale **a** e crea un **vettore** di lunghezza 1. Poi inverte la sua direzione orizzontale e restituisce il nuovo angolo risultante. Ora creiamo la funzione field limits:

```

void State::field_limits ()
{
    if (paddle.pos.x < 0.0)
        paddle.pos.x = 0.0;
    if (paddle.pos.x + paddle.size.x > window_width)
        paddle.pos.x = window_width - paddle.size.x;

    if (ball.pos.x <= 0.0)

```

```

        ball.angle = reflect_horizontal (ball.angle);
    if (ball.pos.x + ball.radius * 2.0 >= window_width)
        ball.angle = reflect_horizontal (ball.angle);
    if (ball.pos.y <= 0.0)
        ball.angle = reflect_vertical (ball.angle);

    if (ball.pos.y + ball.radius * 2.0 >= window_height - paddle_size.y)
    // if (ball.pos.y + ball.radius * 2.0 >= window_height - paddle_size.y * 3.0)
        ball.angle = reflect_vertical (ball.angle);
}

```

Il blocco della racchetta (prime due if):

- **Sinistra:** se provi a muovere la racchetta oltre il bordo sinistro dello schermo il codice blocca a forza sulla coordinata 0.0
- **Destra:** se provi a uscire dal bordo destro allora viene bloccata nel margine.

I muri laterali(le successive due if): Qui controlliamo la palla. Se tocca il muro di sinistra (0.0) o il muro destra (x + il diametro della palla superano la larghezza della finestra) allora applichiamo **reflect_horizontal**.

Il soffitto: se la palla sale troppo e tocca la cima dello schermo chiamiamo **reflect_vertical**.

Il fondo: Controlliamo il pavimento, se la palla scende giù fino ad arrivare all'altezza in cui si trova la racchetta (altezza_finestra - altezza_racchetta), rimbalza di nuovo verso l'altro con **reflect_vertical**.

Aggiungiamo:

```

void State::collisions ()
{
    field_limits ();
}

```

E chiamiamo collision dentro all'update. Quello che dobbiamo fare ora è rendere più robusto il controllo dei limiti. Dobbiamo semplicemente sperare i vector, quindi al posto di lasciarlo dentro al reflect separiamo il codice:

```

sf::Angle reflect_horizontal (sf::Vector2f v)
{
    v.x = -v.x;
    return v.angle ();
}

sf::Angle reflect_vertical (sf::Vector2f v)
{
    v.y = -v.y;
}

```

```

        return v.angle ();
    }

    sf::Angle reflect_horizontal (sf::Angle a)
    {
        sf::Vector2f v (1.0, a);
        return reflect_horizontal (v);
    }

    sf::Angle reflect_vertical (sf::Angle a)
    {
        sf::Vector2f v (1.0, a);
        return reflect_vertical (v);
    }

```

E cambiamo field limits:

```

void State::field_limits ()
{
    if (paddle.pos.x < 0.0)
        paddle.pos.x = 0.0;
    if (paddle.pos.x + paddle.size.x > window_width)
        paddle.pos.x = window_width - paddle.size.x;

    sf::Vector2f v (1.0, ball.angle);
    if (ball.pos.x <= 0.0 && v.x < 0)
        ball.angle = reflect_horizontal (v);
    if (ball.pos.x + ball.radius * 2.0 >= window_width && v.x > 0)
        ball.angle = reflect_horizontal (v);
    if (ball.pos.y <= 0.0 && v.y < 0)
        ball.angle = reflect_vertical (v);
    if (ball.pos.y + ball.radius * 2.0 >= window_height - paddle_size.y * 3.0 && v.y
> 0)
        ball.angle = reflect_vertical (ball.angle);
}

```

Andiamo ad aggiungere a Paddle un metodo Booleano hit che prende in input una Ball e dice se questa interseca la racchetta.

```

bool Paddle::hit(Ball &ball)
{
    sf::Vector2f v(1.0, ball.angle);
    return v.y > 0 &&
        ball.pos.y + ball.radius * 2.0 >= window_height - size.y &&
        ball.pos.x + ball.radius * 2.0 > pos.x &&
        ball.pos.x < pos.x + size.x;
}

```

La **direzione del movimento** è data da **$v.y > 0$** . Verifica che la pallina si stia muovendo verso il basso. Allineamento verticale è dato da **$ball.pos.y + ball.radius * 2.0 \geq window_height - size.y$** . La somma calcola la coordinata Y del bordo inferiore della pallina (posizione + diametro). La differenza calcola la coordinata Y della faccia superiore della racchetta, ritorna true solo se la parte bassa della pallina ha raggiunto o superato l'altezza della racchetta.

Bordo destro è **$ball.pos.x + ball.radius * 2.0 > pos.x$** mentre bordo sinistro è **$ball.pos.x < pos.x + size.x$** .

Aggiungiamo a State un metodo restart, la sua implementazione è la seguente:

```
void State::restart()
{
    paddle = Paddle();
    ball = Ball();
    move_paddle_left = false;
    move_paddle_right = false;
    pause = true;
}
```

Modifichiamo collisions:

```
void State::collisions ()
{
    field_limits ();

    if(paddle.hit(ball))
        ball.angle = reflect_vertical(ball.angle);
}
```

E chiamiamo restart in questo blocco:

```
void State::field_limits ()
{
    // paddle
    if (paddle.pos.x < 0.0) // left
        paddle.pos.x = 0.0;
    if (paddle.pos.x + paddle.size.x > window_width) // right
        paddle.pos.x = window_width - paddle.size.x;

    sf::Vector2f v(1.0, ball.angle);

    // left and right walls
    if (ball.pos.x <= 0.0 && v.x < 0)
        ball.angle = reflect_horizontal(v);
}
```

```

    if (ball.pos.x + ball.radius * 2.0 >= window_width && v.x > 0)
        ball.angle = reflect_horizontal(v);

    // "roof"
    if (ball.pos.y <= 0.0 && v.y < 0)
        ball.angle = reflect_vertical(v);

    // bottom
    if (ball.pos.y > window_height)
        this->restart();
}

```

Aggiungiamo una costante globale `sf::Angle left_max_angle` che poniamo a -170 gradi per fissare un limite all'angolo di riflessione. Aggiungiamo a Paddle un metodo `void strike(Ball&)` per la gestione del rimbalzo; sostituiamo nella `State::collisions` le istruzioni di gestione della collisione con una chiamata a questo metodo.

```

void State::collisions ()
{
    field_limits ();
    paddle.strike(ball);
    // if(paddle.hit(ball))
    //     ball.angle = reflect_vertical(ball.angle);
}

```

Dobbiamo creare il metodo `strike`:

```

void Paddle::strike (Ball& ball)
{
    if (!hit (ball))
        return;

    float ball_center_x = ball.pos.x + ball.radius;
    float paddle_center_x = pos.x + size.x / 2.0;
    float relative_normalized_distance =
        (ball_center_x - paddle_center_x) / (size.x + ball.radius * 2.0) + 0.5;
    sf::Angle ball_right_max_angle = reflect_horizontal (ball_left_max_angle);

    float new_angle_radians =
        linear_interpolation (ball_left_max_angle.asRadians (),
                               ball_right_max_angle.asRadians (),
                               relative_normalized_distance);
    sf::Angle new_angle = sf::radians (new_angle_radians);
    ball.angle = new_angle;
}

```

Se la funzione hit determina che non c'è stato alcun impatto, l'esecuzione si interrompe immediatamente per non sprecare calcoli.

relative_normalized_distance qui stiamo calcolando la distanza tra il centro della pallina e il centro del paddle. Questo valore viene normalizzato aggiungendo +0.5. Funzionerà così:

- **0.0:** impatto sull'estremo bordo sinistro
- **0.5:** impatto al centro perfetto del paddle
- **1.0:** impatto sull'estremo bordo destro

La funzione helper è la seguente:

```
float linear_interpolation (float v0, float v1, float t)
{
    return (1 - t) * v0 + t * v1;
}
```

Aggiungiamo a strike le seguenti righe:

```
float ball_center_y = ball.pos.y + ball.radius;
float paddle_center_y = pos.y + size.y / 2.0;
if (ball_center_y > paddle_center_y)
    new_angle = reflect_vertical (new_angle);
```

qui vengono calcolate le coordinate esatte del centro della pallina e del centro del paddle lungo l'asse verticale, questo serve per quando colpisce i lati della paddle. Dobbiamo rendere il gioco più user friendly, aggiungiamo il feedback di distruzione.

Modifichiamo la struct di Block e Wall in questo modo:

```
struct Block
{
    sf::Vector2f pos;
    sf::Vector2f size;
    bool intact;

    Block (sf::Vector2f pos, sf::Vector2f size) : pos (pos),
                                                size (size),
                                                intact (true) {}

    void draw (sf::RenderWindow& window);
    void hit(Ball& ball);
    bool is_inside(sf::Vector2f p);
};

struct Wall
{
    std::vector<Block> blocks;
```

```

Wall ();
void draw (sf::RenderWindow& window);
void hit(Ball& ball);
};

```

Il metodo hit di Wall si implementa così:

```

void Wall::hit(Ball& ball)
{
    for(auto& block : blocks)
    {
        if(block.intact){
            block.hit(ball);
        }
    }
}

```

Il metodo hit di Ball invece:

```

void Block::hit (Ball& ball)
{
    sf::Vector2f center = {
        ball.pos.x + ball.radius,
        ball.pos.y + ball.radius
    };
    sf::Vector2f north = {
        center.x, center.y - ball.radius
    };
    sf::Vector2f east = {
        center.x + ball.radius, center.y
    };
    sf::Vector2f south = {
        center.x, center.y + ball.radius
    };
    sf::Vector2f west = {
        center.x - ball.radius, center.y
    };

    sf::Vector2f v (1.0, ball.angle);
    if (is_inside (north) && v.y < 0) {
        ball.angle = reflect_vertical (v);
        intact = false;
    }
    if (is_inside (east) && v.x > 0) {
        ball.angle = reflect_horizontal (v);
    }
}

```

```

        intact = false;
    }
    if (is_inside (south) && v.y > 0) {
        ball.angle = reflect_vertical (v);
        intact = false;
    }
    if (is_inside (west) && v.x < 0) {
        ball.angle = reflect_horizontal (v);
        intact = false;
    }
}

```

Il codice calcola il centro della pallina e i suoi quattro punti cardinali. Questi punti vengono usati come “sensori” per capire quale lato della pallina ha toccato il blocco. **North/South** sommano/sottraggono il raggio alla coordinata Y.

East/West sommano/sottraggono il raggio alla coordinata X.

Viene creato un vettore *v* che rappresenta la direzione attuale della pallina basata sul suo angolo → **sf::Vector2f v (1.0, ball.angle) ;**

Il codice esegue quattro controlli if, uno per ogni direzione. Per ogni punto (es. north):

1. **is_inside(point):** Controlla se quel punto specifico della pallina si trova dentro il perimetro del blocco.
2. **Direzione:** Verifica che la pallina si stia muovendo *verso* il blocco (es. *v.y* < 0 per il punto nord) per evitare bug di incastro.
3. **Riflessione:** Se c'è collisione:
 - L'angolo della pallina viene cambiato tramite `reflect_vertical` o `reflect_horizontal`.
 - `intact = false`: Il blocco viene segnato come "distrutto".

Infine l'ultima funzione da implementare è questa:

```

bool Block::is_inside (sf::Vector2f p)
{
    return

    p.x >= pos.x &&

    p.y >= pos.y &&

    p.x <= pos.x + size.x &&

```

```
p.y <= pos.y + size.y;
```

```
}
```